



Sparse oblique decision trees: a tool to understand and manipulate neural net features

Suryabhan Singh Hada¹ · Miguel Á. Carreira-Perpiñán¹ ·
Arman Zharmagambetov¹

Received: 1 April 2021 / Accepted: 5 October 2022 / Published online: 9 January 2023

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2023

Abstract

The widespread deployment of deep nets in practical applications has lead to a growing desire to understand how and why such black-box methods perform prediction. Much work has focused on understanding what part of the input pattern (an image, say) is responsible for a particular class being predicted, and how the input may be manipulated to predict a different class. We focus instead on understanding which of the internal features computed by the neural net are responsible for a particular class. We achieve this by mimicking part of the neural net with an oblique decision tree having sparse weight vectors at the decision nodes. Using the recently proposed Tree Alternating Optimization (TAO) algorithm, we are able to learn trees that are both highly accurate and interpretable. Such trees can faithfully mimic the part of the neural net they replaced, and hence they can provide insights into the deep net black box. Further, we show we can easily manipulate the neural net features in order to make the net predict, or not predict, a given class, thus showing that it is possible to carry out adversarial attacks at the level of the features. These insights and manipulations apply globally to the entire training and test set, not just at a local (single-instance) level. We demonstrate this robustly in the MNIST and ImageNet datasets with LeNet5 and VGG networks.

Keywords Decision trees · Deep neural networks · Interpretability

Responsible editor: Martin Atzmueller, Johannes Fürnkranz, Tomáš Kliegr and Ute Schmid

✉ Suryabhan Singh Hada
shada@ucmerced.edu

✉ Miguel Á. Carreira-Perpiñán
mcarreira-perpinan@ucmerced.edu

Arman Zharmagambetov
azharmagambetov@ucmerced.edu

¹ University of California, Merced, USA

1 Introduction

Deep neural nets are accurate black-box models. They are highly successful in terms of predictive performance (say, classifying an input image) but remarkably difficult to understand in terms of how exactly they come up with a prediction for an input. Both of these issues have been known to researchers and practitioners for many years, but it is in the 2010s that deep learning has achieved a wild, unexpected success that has attracted widespread attention beyond computer science. In only a few years, neural nets have become the workhorse model in a number of practical problems, in computer vision, speech and language processing, games, self-driving cars and other engineering applications; legal, financial and medical applications; and many others. Neural nets now underlie intelligent processing in desktops, cloud computing and IoT devices. Yet, the way neural nets are defined and optimized, and the sheer size and complexity of state-of-the-art deep nets, make them very hard to understand in explanatory terms. This is also true of other machine learning models, but deep nets sit at the far end in terms of opaqueness. Deep neural nets are generally not based on mechanistic models that involve physical entities in a causal way. They purely learn a correspondence between complex high-dimensional inputs and outputs by means of function approximation techniques based on using many adjustable building blocks (layers, neurons, weights and various nonlinear transformations). Such models can potentially approximate many possible correspondences with appropriate choices for these parameters, and finding a good choice is possible via a numerical optimization algorithm that minimizes a prediction loss over a large, labeled dataset. The resulting net can make highly accurate predictions for test inputs, but leaves many questions unanswered. We do not know what a given neuron or weight, or group of them, codes for at a level that a human can understand; or what would happen if we remove or alter a given set of neurons or weights; or what should we change in the input instance to change the prediction in a certain way; or what should we change in the trained net to correct a wrong prediction in some specific input instance. Further, for reasons not well understood, the class predicted by a deep net can be very sensitive to minute alterations of the input in ways that can be used adversarially.

Most of these questions are not new (Guidotti et al. 2018; Rudin 2019), but they have become urgent due to the widespread deployment of deep nets in sensitive applications. Indeed, this is leading to law changes regarding the use of AI systems and data, such as the EU General Data Protection Regulation. Erring occasionally in a movie recommendation system is not serious, but crafting an adversarial attack so it systematically recommends or does not recommend certain movies is. Much more serious are errors or attacks in legal, financial or medical applications. An example of all these three at once is the processing of medical insurance claims, an area traditionally fraught with more or less legal attempts to affect the payment outcomes. The use of deep nets for automatic claim decision making introduces further opportunities for manipulation that are more creative and difficult to detect (Finlayson et al. 2019). More generally, there is a need to understand AI systems experimentally, and different perspectives (including but not limited to computer science) will likely be necessary (Rahwan et al. 2019).

Our paper has two contributions that can improve our ability to explain and manipulate trained deep nets. Firstly, we propose sparse oblique decision trees as a tool to understand deep nets. Using decision trees is by itself not a new idea. What is new is the specific, novel type of tree we use, and how we apply it to a given deep net. Traditional tree learning algorithms typically construct trees where each decision node thresholds a single input feature (axis-aligned trees). Although such trees are considered among the most interpretable machine learning models, this is only true if the tree is relatively small; it is very hard to interpret an axis-aligned tree with thousands of nodes. More importantly, the axis-aligned decisions are ill-suited to handle datasets with many, correlated features. In practice, axis-aligned trees usually achieve too low an accuracy, and are wholly inadequate for high-dimensional complex inputs such as pixels of an image or neural net features. We capitalize on a recently proposed *Tree Alternating Optimization (TAO)* algorithm which can learn far more accurate trees that remain small and very interpretable, because each decision node operates on a small, learnable subset of features.

Second, we apply the tree to an internal layer of the deep net, hence mimicking its remaining (classifier) layers, rather than attempting to mimic the entire deep net. This allows us to probe the relation between deep net features and classes. As a subproduct, inspection of the tree allows us to construct a new kind of adversarial attacks where we manipulate the deep net features via a mask to block a specific set of neurons. This gives us surprising control on what class the deep net will output. Among other possibilities, we can make it output the same, desired class for all dataset instances; or make it never output a given class; or make it misclassify certain pairs of classes.

Next, we review related work (Sect. 2) and the TAO algorithm (Sect. 3), describe how we use trees to understand and manipulate deep net features (Sect. 4, 5), and demonstrate this in MNIST and ImageNet data with LeNet5 and VGG16 deep neural nets (Sect. 6). A short version of this paper appeared in Hada et al. (2021).

2 Related work

The last few years have seen much work in the area of interpreting or understanding, in some way, the internal workings of a trained neural network. We describe the most relevant work, organized in several categories.

2.1 Feature inversion or activation maximization

The idea here is to find what input feature vectors (e.g. what images, for a VGG16 network) produce a certain output under the neural network. This can be done for individual neurons, with the goal of understanding what “concept” a neuron may encode, or for an entire layer of neurons. Mathematically, this is essentially a problem of inverting the neural net function.

One way to do this is to formulate the inversion problem as an optimization: to minimize the Euclidean distance between the target outputs (at a neuron or layer of neurons) and the outputs generated by the input feature vector sought. In its basic form,

this idea goes back decades (Kindermann and Linden 1990; Jensen et al. 1999), and has been revisited by various papers recently. With images, naive application of this procedure will generate noisy or unrealistic images. Various approaches have been proposed to mitigate this, such as regularizing the optimization problem using total variation (Mahendran and Vedaldi 2016) or data-driven patch priors (Wei et al. 2015), or learning these regularizers by training a deep neural network to generate images from the features (Dosovitskiy and Brox 2016).

Another way is to seek an input pattern that will maximize the activation (output) of a given neuron, analogously to seeking the “receptive field” of the neuron. Such pattern would represent the preferred stimulus to which that neuron responds, and might give a clue to what that neuron encodes. This is again an optimization problem over the input space of the network, which can be solved in various ways (e.g. (Simonyan et al. 2014)). Other works seek to provide multiple patterns rather than a single one, in order to find a better characterization of a given neuron (Nguyen et al. 2016, 2017; Hada et al. 2019). Again, this can be combined with regularizers or generative adversarial networks to obtain realistic images.

Finally, there also exist approaches that are not based on optimization (Bau et al. 2017; Mu and Andreas 2020).

2.2 Local, instance-level explanations

This line of work seeks to explain the neural net prediction for a given input instance (or group of instances). For example: what part of a given input image was mostly responsible for the network to classify it as a certain class? This is often referred to as a *saliency map* of the image. An intuitive way to do this is via sensitivity analysis, such as computing or approximating the gradient of the output score with respect to the input image (Simonyan et al. 2014; Zeiler and Fergus 2014). With ReLU activation functions, which have a discontinuous gradient, this can produce artifacts (Sundararajan et al. 2017; Shrikumar et al. 2017). Other approaches and variations exist (Bach et al. 2015; Montavon et al. 2016, 2018; Shrikumar et al. 2017; Zhou et al. 2016; Selvaraju et al. 2017; Fong and Vedaldi 2017; Qi et al. 2020; Sundararajan et al. 2017). Although saliency maps are visually appealing and can sometimes agree with human intuition, it is not clear how robust and consistent they are, and they can actually be misleading depending on the case (Fong and Vedaldi 2017; Adebayo et al. 2018; Ghorbani et al. 2019; Rudin 2019). For example, the saliency map for a given image may be very similar even when it is computed for different target classes.

Another way to seek input features that are particularly important in predicting a given instance’s class are *Shapley values*, originally proposed to attribute reward among players of a cooperative game. Calculating Shapley values is NP-hard, so they are approximated in practice (Štrumbelj and Kononenko 2014; Datta et al. 2016; Merrick and Taly 2020; Lundberg and Lee 2017). The ability of Shapley values to provide explanations that are useful for humans has also been criticized (Kumar et al. 2020).

Explanation by examples is another way to provide instance-level explanations, where we seek which instances in the training set (on which the neural net was trained)

are most responsible for a given instance to be classified as a certain class. Naively, this would involve retraining the neural net without each particular instance, to see what the effect of that instance would have been. This is computationally very costly and is approximated in various ways (Koh and Liang 2017; Yeh et al. 2018; Pruthi et al. 2020).

Finally, another line of work for local explanation is to replace the neural network function locally around the given instance with a simpler model that can be interpreted. One of these methods is LIME (Ribeiro et al. 2016), which describes a somewhat involved procedure to fit a sparse linear model using a sample of instances near the given instance. The nonzero coefficients in this linear model can be used to gauge the importance of input features in the prediction for the instance. This has been extended to decision rules (Ribeiro et al. 2018) instead of a linear model. Some of these local explanation methods can be seen from a common point of view of explaining the network's output as a weighted sum of the input features (Lundberg and Lee 2017). Other works (Singh et al. 2019; Zhang et al. 2019) are based on constructing an agglomerative clustering tree over the neurons or input features. This is done by defining a similarity measure for the latter in terms of their ability to predict the local instance. The clustering tree provides a hierarchical arrangement of the input features and can be inspected by a human to look for groups of input features that are influential in the original instance's prediction. (Although the clustering tree is called a "decision tree" in (Zhang et al. 2019), it is not a decision tree in the usual sense of classification or regression.) Because of the multiple approximations involved and the lack of a clear criterion of what the proxy model is supposed to explain, these approaches are somewhat ad-hoc.

2.3 Global explanation via an interpretable mimic of the neural network

The goal of these types of methods is to mimic the entire deep neural net via a more interpretable model such as decision trees or decision rules. This then provides a *global explanation*, applicable to any input instance, unlike the previous, local explanation methods, which are only valid near a given instance.

The topic of extracting sets of rules from neural nets was actively researched in the 1990s (Andrews et al. 1995; McCormick et al. 2013; Guidotti et al. 2018). Two basic approaches were used: in rule extraction as search (Fu 1994; Towell and Shavlik 1993), a specialized heuristic search over possible rules was based on the neurons' connectivity pattern, but this assumed binary activations and did not scale beyond small nets. In rule extraction as learning, or the teacher-student approach (Craven and Shavlik 1994, 1996; Domingos 1998), one trains a decision tree to mimic the neural net by using the latter's predictions on the training set (possibly augmented with random instances). Crucially, the success of this idea relies on the faithfulness of the mimic. Although some of these papers (Towell and Shavlik 1993; Craven and Shavlik 1996; Baesens et al. 2003) claimed that the extracted rules closely approximate the original neural net, this was based on very small problems and networks (single-layer). In such problems, training an axis-aligned decision tree directly was not far in accuracy

from the neural net in the first place, and could produce a relatively small tree and a correspondingly small set of rules.

The fundamental problem with this is that traditional algorithms to learn decision trees or decision rules, such as CART (Breiman et al. 1984) or C4.5 (Quinlan 1993), based on axis-aligned trees, are unable to learn accurate enough trees to be useful mimics of a neural net except in very small, low-dimensional problems. They fall far short of handling the large, deep neural nets that are used in current computer vision applications, for example.

2.4 Our work in context

Our work belongs to the category of global explanation via an interpretable mimic that is a decision tree, with two important differences. First, we use a special type of decision tree, a *sparse oblique tree*. This can be trained to be much more accurate than axis-aligned, CART-type trees, which makes it more likely that one can obtain a faithful mimic. At the same time, the sparse oblique tree remains interpretable. This is because the tree size is far smaller than that of an axis-aligned tree (in depth and number of nodes), and because it uses relatively few features in each decision node. As we show in our experiments, inspection (manual or automatic) of the nonzero weights in the decision nodes leads to important insights about the tree (and about the neural net), and shows how to manipulate the neural net features to alter the classification result in a desired way. (Further in terms of interpretability, counterfactual explanations can be solved exactly and efficiently for oblique trees (Carreira-Perpiñán and Hada 2021; Hada et al. 2021), although we do not use this here.)

A second difference is that we do not aim at replacing the entire neural net, but at replacing the classifier portion of a deep net (globally over all instances). *This allows us to study the relation between the deep net features (neuron activations at a certain internal layer) and the output classes*—note that those features were specifically learned by the neural net during training with the goal of predicting the classes optimally. This is unlike much of the work cited earlier, which studies the relation between input features (e.g. pixels) and neuron activations at a certain layer. (In Zharmagambetov et al. (2021), we also train a sparse oblique tree on features obtained from multiple pretrained deep neural networks, but the goal there is not to interpret a neural net, but to achieve a model with higher accuracy than the pretrained networks.)

3 Learning sparse oblique trees with the Tree Alternating Optimization (TAO) algorithm

We briefly explain the Tree Alternating Optimization (TAO) algorithm; the original references give more details (Carreira-Perpiñán 2022; Carreira-Perpiñán and Tavallali 2018). Among other types of trees, TAO can learn *sparse oblique trees*. These have a constant label at each leaf and a linear decision function at each decision node but, crucially, the decision function only uses a typically small, learned subset of

features (see Fig. 5). TAO achieves this by optimizing an objective function which is the sum of the classification loss and an ℓ_1 penalty (with hyperparameter $\lambda \geq 0$) on the weight vectors of the decision nodes (similar to a LASSO (Hastie et al. 2015) but on each decision node). Each TAO iteration decreases this objective and consists of optimizing over groups of non-descendant nodes (such as all the nodes at the same depth), where the objective can be shown to separate over the nodes in the group. The optimization over each node can be shown to be equivalent to a simpler, “reduced” problem taking the form of a majority classifier at a leaf, and a binary classifier at a decision node. In the latter, each instance reaching the node is assigned a “pseudolabel” indicating the child that gives the lowest loss under the current tree. TAO uses a fixed tree structure while iterating, which is automatically pruned because subtrees may eventually receive training instances of the same class, or not receive instances at all; this is heavily promoted by the ℓ_1 penalty on the decision nodes, which can drive all weights in a decision node to zero, thus making it redundant.

In more detail, consider a tree T of a given structure (usually, complete of depth Δ) with nodes in a set $\mathcal{N}$ and parameters $\Theta = \{\theta_i\}_{i \in \mathcal{N}}$. For a decision node i , θ_i consists of the weights and bias of the hyperplane. For a leaf i , $\theta_i \in \{1, \dots, K\}$ is a class label. Now, to train T on a dataset $\{\mathbf{x}_n, y_n\}_{n=1}^N \subset \mathbb{R}^D \times \{1, \dots, K\}$ of input instances and their labels, we optimize the following objective function:

$$\min_{\Theta} E(\Theta) = \sum_{n=1}^N \mathcal{L}(y_n, T(\mathbf{x}_n; \Theta)) + \lambda \sum_{i \in \mathcal{N}} \phi_i(\theta_i) \quad (1)$$

where $\mathcal{L}(\cdot, \cdot)$ is the cross-entropy loss, $\phi_i(\theta_i)$ is a regularization term with hyperparameter $\lambda \geq 0$ (we will use ℓ_1 regularization on the weight vectors of the decision nodes). TAO relies on two theorems: a separability condition, and a reduced problem over each node (see details and proofs in Carreira-Perpiñán (2022), Carreira-Perpiñán and Tavallali (2018)). Here, we describe them briefly.

Theorem 31 (separability condition). *Consider any pair of nodes i and j . Assume the parameters of all other nodes (Θ_{rest}) are fixed. If nodes i and j are not descendants of each other, then $E(\Theta)$ can be rewritten as:*

$$E(\Theta) = E(\theta_i) + E(\theta_j) + E(\Theta_{\text{rest}}). \quad (2)$$

In other words, the separability condition states that any set of non-descendant nodes of a tree can be optimized independently. Note that $E(\Theta_{\text{rest}})$ can be treated as a constant since we fix Θ_{rest} .

Theorem 32 (reduced problem). *For a single node i , optimizing over its parameters simplifies to a well-defined reduced problem over the instances that currently reach node i (the reduced set $\mathcal{R}_i \subset \{1, \dots, N\}$).*

- For a decision node i , the reduced problem can be written as a 0/1 loss binary classification problem, where the target class refers to the left or right subtree. For this we assign a “pseudolabel” ($\bar{y} \in \{\text{left}, \text{right}\}$) indicating the child that

gives the lowest loss under the current tree. Thus, the reduced problem takes the form:

$$E(\theta_i) = \sum_{n \in \mathcal{R}_i} \bar{L}_n(\bar{y}_n, (f(\mathbf{x}_i); \theta_i)) + \lambda \phi_i(\theta_i) \quad (3)$$

where $\bar{L}_n$ is the 0/1 loss and $f_i: \mathbb{R}^D \rightarrow \{\text{left}, \text{right}\}$ is the decision function at node i with parameters θ_i (which, for an oblique tree, is defined by a linear classifier). This is an NP-hard problem, but it can be approximated by using a convex surrogate loss. In this work, we use ℓ_1 regularized logistic regression ($\phi_i = \|\cdot\|_1$) and optimize it using LIBLINEAR (Fan et al. 2008).

- For a leaf node i , the reduced problem can be shown to have an exact solution, given by setting the leaf label θ_i to the majority class among all the instances in its reduced set $\mathcal{R}_i$.

The above theorems mean that we can monotonically reduce the objective function (1) by cycling over the nodes in the tree in some order and optimizing each node's parameters by solving its reduced problem. We can optimize any subset of non-descendant nodes in parallel, e.g. all nodes at the same depth. As for the tree structure (and initial parameters), this will depend on the dataset, but we find that using a complete tree of large enough depth Δ and random (Gaussian 0,1) initial parameters often works well.

In a sense, TAO operates similarly to how a neural net (or other machine learning models) are trained: by fixing the model structure and optimizing a desired loss function iteratively. However, instead of using gradients (which are not available for a decision tree), it uses alternating optimization. Model selection over the tree structure happens automatically by using a large enough tree structure and letting TAO prune it via the ℓ_1 penalty (as has also been done to prune weights and neurons in neural nets, e.g. (Carreira-Perpiñán and Idelbayev 2018)). We describe the entire tree training process in Fig. 1. As described in Carreira-Perpiñán (2022); Carreira-Perpiñán and Tavallali (2018), the computational complexity of one TAO iteration is upper bounded by the depth of the tree times the cost of solving a logistic regression problem on the entire dataset. This is because the reduced sets of all the nodes at the same depth form a partition of the entire training set.

We can control the tradeoff between accuracy and sparsity (in terms of the size of the tree and the number of nonzero weights at the nodes), and hence control the amount of feature selection and interpretability, similarly as with the LASSO regularization path (Hastie et al. 2015). We simply trace a family of trees of decreasing accuracy and increasing sparsity as the hyperparameter λ goes from 0 to ∞ .

TAO considerably improves (Zharmagambetov et al. 2021) over the traditional, widely used algorithms (such as CART (Breiman et al. 1984) or C4.5 (Quinlan 1993)) that are based on greedy recursive partitioning based on a purity measure. This is because these algorithms have no useful guarantees concerning the classification loss to start with, and are only moderately effective with axis-aligned trees. The same approach can be used to train oblique trees (Breiman et al. 1984; Murthy et al. 1994), but it results in large, nonsparse and highly suboptimal trees that often do not improve over axis-aligned ones. These are the fundamental reasons why axis-aligned trees are


```

input training set  $\{(\mathbf{x}_n, y_n)\}_{n=1}^N \subset \mathbb{R}^D \times \{1, \dots, K\}$ 
        initial tree  $T$ 
repeat
  for  $i \in$  nodes of  $T$ , visited in reverse BFS
    if  $i$  is a leaf then
       $\theta_i \leftarrow$  majority-class label in the reduced set  $\mathcal{R}_i$ 
    else
      generate pseudolabels  $\bar{y}_n$  for each instance  $n \in \mathcal{R}_i$ 
       $\theta_i \leftarrow$  minimizer of the reduced problem (eq. (3))
  until stop
postprocess  $T$ : remove dead branches & pure subtrees
return  $T$ 

```

Fig. 1 Pseudocode for the tree alternating optimization (TAO) algorithm (Carreira-Perpiñán 2022; Carreira-Perpiñán and Tavallali 2018). Visiting each node in reverse breadth-first search (BFS) order means scanning depths from $\text{depth}(T)$ down to 1, and at each depth processing (in parallel, if so desired) all nodes at that depth. “stop” occurs when either the objective function decreases less than a set tolerance or the number of iterations reaches a set limit

the only type of tree that is widespread in practice, at least until now. TAO also improves tree ensembles (forests) considerably: if using TAO instead of a CART-type algorithm (as done in random forests (Breiman 2001) and XGBoost (Chen and Guestrin 2016)), the resulting forest contains fewer, shallower trees but is consistently more accurate, whether using bagging or boosting to ensemble the trees (Carreira-Perpiñán and Zharmagambetov 2020; Zharmagambetov et al. 2020, 2021), Zharmagambetov et al. (2021). Finally, TAO can also be used to train novel forms of tree models (Zharmagambetov et al. 2021), Zharmagambetov et al. (2021).

4 Sparse oblique trees: a microscope to observe a deep neural net

Our overall approach is as follows (see Fig. 2). Assume we have a trained deep net $\mathbf{y} = \mathbf{f}(\mathbf{x})$ for classification of an input instance $\mathbf{x} \in \mathbb{R}^D$ into K classes, so $\mathbf{y}$ is a vector of K softmax values encoding (an approximation to) the class distribution given $\mathbf{x}$. We write the net $\mathbf{f}(\mathbf{x}) = \mathbf{g}(\mathbf{F}(\mathbf{x}))$ as the composition of a feature-extraction layer $\mathbf{F}$, so $\mathbf{z} = \mathbf{F}(\mathbf{x}) \in \mathbb{R}^F$ are the deep net features (neuron outputs at that layer), and a classifier layer $\mathbf{y} = \mathbf{g}(\mathbf{z})$ consisting of the rest of the net.¹ This includes as particular cases of features the raw inputs $\mathbf{x}$ (where $\mathbf{F}$ is the identity) and the class label or softmax outputs (where $\mathbf{g}$ is the identity), but we will usually be more interested in features at an intermediate layer. Each neuron at that layer can be considered as a feature detector which encodes some property or concept of the input pattern $\mathbf{x}$, which may be useful (in combination with other neurons’ concepts) to provide information for or against one or more classes.

¹ This is an operational definition, since the original net was trained without an explicit construction of feature extraction and classifier, and indeed such distinction is blurred in some architectures such as ResNets (He et al. 2016). Other architectures, such as LeNet5 (LeCun et al. 1998) and VGG (Simonyan and Zisserman 2015) do have a clear separation into feature extraction (based on convolutional, pooling and subsampling layers), and classification (fully-connected MLP).

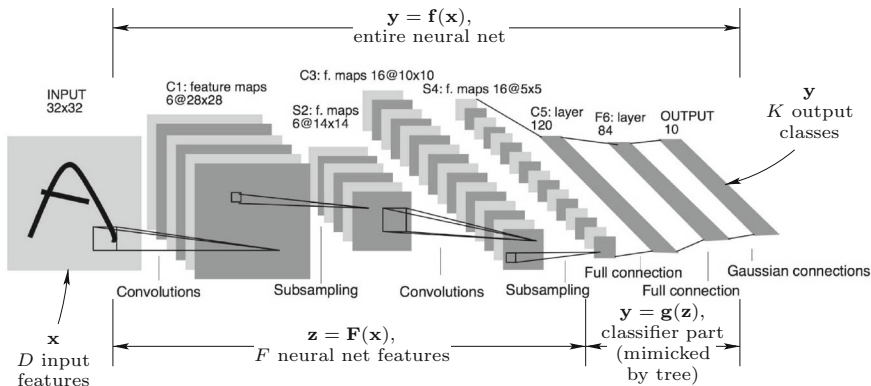


Fig. 2 Mimicking part of a neural net with a decision tree. The figure shows the neural net $y = f(x) = g(F(x))$, considered as the composition of a feature extraction part $z = F(x)$ and a classifier part $y = g(z)$. For example, for the LeNet5 neural net of (LeCun et al. 1998) in the diagram, this corresponds to the first 4 layers (convolutional and subsampling) followed by the last 2, fully-connected layers, respectively. The “neural net feature” vector z consists of the activations (outputs) of F neurons, and can be considered as features extracted by the neural net from the original features x (pixel values, for LeNet5). We use a sparse oblique tree to mimic the classifier part $y = g(z)$, by training the tree using as input the neural net features z and as output the corresponding ground-truth labels

Assume we have a dataset (usually the one used to train the net)

$\{(\mathbf{x}_n, y_n)\}_{n=1}^N \subset \mathbb{R}^D \times \{1, \dots, K\}$ of input instances and their labels. Then:

1. Train a sparse oblique tree $y = T(z)$ with TAO on the training set $\{(F(\mathbf{x}_n), y_n)\}_{n=1}^N \subset \mathbb{R}^F \times \{1, \dots, K\}$. Explore the interpretability-accuracy tradeoff over a useful range of the sparsity hyperparameter $\lambda \in [0, \infty)$ and pick a final tree. Usually this will be a tree with close to highest validation accuracy and as sparse as possible.
2. Inspect the tree to find interesting patterns about the deep net.

Our goal is to achieve a tree that both mimics well the deep net and is as simple as possible. We achieve this by training the tree on the same training set as the net (using the latter’s features but the ground-truth labels²).

Step 2 is purposely vague. There is probably a wealth of information in the tree regarding the features’ meaning and effect on the classification, both at the level of a specific input instance or more globally. In this paper we focus on one specific pattern described in the next section.

5 Manipulating the features of a deep net to alter its classification behavior

Our overall objective is to manipulate the value of the deep net features $z \in \mathbb{R}^F$ to alter in a controlled way the class predicted by the net. We will not alter the weights of the net, i.e., F and g remain the same. We just alter z into a masked $\bar{z} = \mu(z) = \mu^\times \odot z + \mu^+$

² This is equivalent to using the deep net predictions as labels (teacher-student approach), because our deep nets achieve nearly zero training error.

via a *multiplicative and an additive mask* $\mu^\times, \mu^+ \in \mathbb{R}^F$, respectively (where “ $\odot$ ” means elementwise multiplication). Specifically, we have:

$$\text{Original net: } \mathbf{y} = \mathbf{f}(\mathbf{x}) = \mathbf{g}(\mathbf{F}(\mathbf{x})) \quad (4)$$

$$\text{Original features: } \mathbf{z} = \mathbf{F}(\mathbf{x}) \quad (5)$$

$$\text{Masked net: } \bar{\mathbf{y}} = \bar{\mathbf{f}}(\mathbf{x}) = \mathbf{g}(\mu(\mathbf{F}(\mathbf{x}))) \quad (6)$$

$$\text{Masked features: } \bar{\mathbf{z}} = \mu(\mathbf{F}(\mathbf{x})) = \mu(\mathbf{z}). \quad (7)$$

We demonstrate the masking operation in Fig. 3. In the simplest, most intuitive version of the mask, we just need a binary multiplicative mask $\bar{\mathbf{z}} = \mu^\times \odot \mathbf{z}$ where $\mu^\times \in \{0, 1\}^F$. Using an additive mask and real-valued masks makes the manipulation’s effect more robust and harder to detect.

We will construct a mask by inspecting the tree, specifically by observing the weight of each feature in each decision node of the tree. *By selectively zeroing some features we can guarantee that any instance will follow a specific child in a given node and hence direct instances as desired towards a target leaf.* Under some assumptions, we will be able to guarantee a desired effect if using the tree, i.e., in the classifier $y = T(\mu(\mathbf{F}(\mathbf{x})))$. Then we will apply the mask to the deep net as $\bar{\mathbf{y}} = \mathbf{g}(\mu(\mathbf{F}(\mathbf{x})))$. While we cannot guarantee anything in the masked net, *we can reasonably expect similar results if the tree is a good mimic of the classifier $\mathbf{g}$, and indeed our experiments show that the masked net behaves like the masked tree most of the times.*

At a decision node i , the decision rule is “if $\mathbf{w}_i^T \mathbf{x} + b_i \geq 0$ then go to right child, else go to left child”, where $\mathbf{w}_i \in \mathbb{R}^F$ is the weight vector and $b_i \in \mathbb{R}$ the bias. We will assume the following (throughout we use elementwise notation as needed, as in “ $\mathbf{z} \geq \mathbf{0}$ ”):

- The deep net features are nonnegative: $\mathbf{z} = \mathbf{F}(\mathbf{x}) \geq \mathbf{0}$. This is true for ReLUs, which are used in most deep nets at present.
- The bias at each decision node i of the tree is zero: $b_i = 0$. This holds very well in the trees we trained, specifically $|b_i| \ll \|\mathbf{w}_i\|$ at each decision node i .

If these assumptions do not hold, it is still possible to design masks that work reliably in some cases. We mention some of them but generally leave such details out of this paper.

5.1 Diverting all instances to one child

We give a basic procedure “ $\mu^\times, \mu^+ \leftarrow \text{NODE-MASK}(i, c)$ ” that underlies our mask construction. Assume an instance $\mathbf{z}$ reaches a decision node i in the tree. NODE-MASK produces a mask that guarantees that $\bar{\mathbf{z}} = \mu^\times \odot \mathbf{z} + \mu^+$ goes left (right) if $c = \text{left}$ (right) child, for any instance $\mathbf{z}$ in the training set. Essentially, NODE-MASK allows us to “cut” a subtree, so by applying it as needed we can cut subtrees to leave only a certain path in the tree for any instance to follow and hence effect a desired classification.

NODE-MASK works as follows. Call the weight vector $\mathbf{w}$ and bias b (assumed zero anyway) at node i . Write them w.l.o.g. as $\mathbf{w} = (\mathbf{w}_0 \ \mathbf{w}_- \ \mathbf{w}_+)$ and $\mathbf{z} = (\mathbf{z}_0 \ \mathbf{z}_- \ \mathbf{z}_+)$ where $\mathbf{w}_0 = \mathbf{0}$, $\mathbf{w}_- < \mathbf{0}$ and $\mathbf{w}_+ > \mathbf{0}$ contain the zero, negative and positive weights in $\mathbf{w}$,

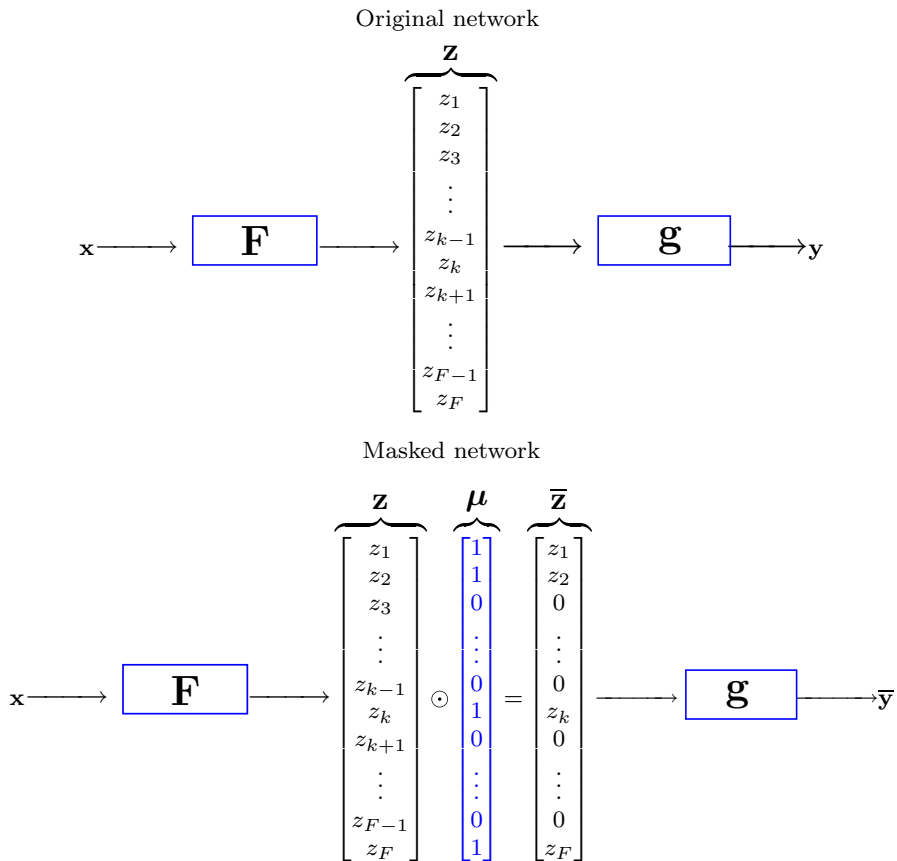


Fig. 3 *Top* original network. *Bottom* masking operation in the network. The symbols' meaning is as follows: input $\mathbf{x}$, feature extraction part of the network $\mathbf{F}$, original features $\mathbf{z}$, binary mask created using the tree $\boldsymbol{\mu}$, modified features $\bar{\mathbf{z}}$, classifier part of the network $\mathbf{g}$, original output $\mathbf{y}$, and modified output $\bar{\mathbf{y}}$

and $\mathbf{z} \geq \mathbf{0}$ is reordered according to that. Call $\mathcal{S}_0$, $\mathcal{S}_-$ and $\mathcal{S}_+$ the corresponding sets of indices in $\mathbf{w}$. Then $\mathbf{w}^T \mathbf{z} + b = \mathbf{w}_-^T \mathbf{z}_- + \mathbf{w}_+^T \mathbf{z}_+$ with $\mathbf{w}_-^T \mathbf{z}_- \leq 0$ and $\mathbf{w}_+^T \mathbf{z}_+ \geq 0$. So if $\mathbf{z}_- = \mathbf{0}$ then $\mathbf{w}^T \mathbf{z} + b \geq 0$ and $\mathbf{z}$ would go to the right child and if $\mathbf{z}_+ = \mathbf{0}$ then $\mathbf{w}^T \mathbf{z} + b \leq 0$ and $\mathbf{z}$ would go to the left child. Hence, our masks are as follows: to go left, $\boldsymbol{\mu}^\times \in \{0, 1\}^F$ is a binary vector containing ones at $\mathcal{S}_-$, zeros at $\mathcal{S}_+$ and $*$ (meaning any value) at $\mathcal{S}_0$; and $\boldsymbol{\mu}^+ \geq \mathbf{0}$ is a vector containing small positive values at $\mathcal{S}_-$ and zero elsewhere. To go right, exchange “ $-$ ” and “ $+$ ” in the procedure. The additive mask $\boldsymbol{\mu}^+$ is necessary only if the features in $\mathcal{S}_-$ all happen to be zero (which would produce $\mathbf{w}^T \bar{\mathbf{z}} + b = 0$ and be on the decision boundary). This is unlikely to happen unless $\mathcal{S}_-$ contains very few features, but still the additive mask is useful to

push $\mathbf{w}^T \bar{\mathbf{z}}$ away from the boundary and hence make it more likely that the masked deep net will perform as desired³.

5.2 Masks

We now show how to construct masks that effect a certain class outcome. For each case, we state the desired goal and the corresponding mask. In the manipulations below we may use NODE- MASK repeatedly over several nodes to construct the mask (which is applied to the feature vector and hence applies globally to each node). In that case, we will only use the multiplicative mask produced by NODE- MASK at each node, and create the additive mask at the end given the final multiplicative mask.

- ALL CLASS k_1 TO CLASS k_2 : let $k_1 \neq k_2 \in \{1, \dots, K\}$. For any instance $\mathbf{x}$ originally classified as k_1 , classify it as k_2 . For any other instance, do not alter its classification. This case only works if the classes k_1 and k_2 are leaf siblings (have the same parent). Class k_2 may be represented by multiple leaves since we only need to deal with one of them (the sibling of k_1). Mask: simply apply NODE- MASK to the parent of the leaves of k_1 and k_2 .
- NONE TO CLASS k : let $k \in \{1, \dots, K\}$. For any instance $\mathbf{x}$ originally classified as k , classify it as any other class. For any other instance, do not alter its classification. Mask: simply apply NODE- MASK to the parent of each leaf of k and combine the resulting multiplicative masks as extended-AND (defined below). Finally, add the additive mask.

Strictly speaking, we can guarantee that class- k instances are classified as some other class, but not that we do not alter the classification of other instances. This is because the features that are masked out may appear in other nodes and possibly affect the path of an instance. However, with our deep nets the number of features masked out is very small and the mask works well. If the features selected in a node only appear in that node, then their effect is purely local, of course.

- ALL TO CLASS k : let $k \in \{1, \dots, K\}$. Classify all instances $\mathbf{x}$ as class k . Mask: find the path from the root to the leaf of class k . At each node i in the path, apply NODE- MASK (to divert instances along the path) and keep the multiplicative mask only. The final multiplicative mask, elementwise, has a 0 where any of the node masks has a 0, a 1 where all node masks have no 0s but at least one 1, and * elsewhere. This masks out all the “undesired” features that might divert us from the path. Equivalently, this is the logical extended-AND of all the multiplicative masks along the path (where we extend AND to mean $\text{AND}(*, 0) = 0$, $\text{AND}(*, 1) = 1$ and $\text{AND}(*, *) = *$).

This would not work if the multiplicative mask is zero at all features, but this is unlikely if the nodes have sparse weight vectors, as happens in our experiments.

This also works if class k is represented as multiple leaves. We simply take the union of the masks over each leaf.

³ In fact, just setting $\mathbf{z} = \boldsymbol{\mu}^+$ (i.e., replacing the features with the additive mask without even using the multiplicative mask) would work in the tree. This boils down to replacing any incoming feature vector with a fixed feature vector of known classification under $\mathbf{g}$. But this makes the attack very obvious: the softmax values outputted by the net are the same for every instance.

- NONE TO A SUBSET OF CLASSES: we can apply this in some cases depending on the tree. It simply follows by applying NODE- MASK to a given decision node. The classes in the subtree that is cut out cannot be reached by any instance. Applying this to multiple nodes and creating the multiplicative mask by extended-AND removes the corresponding subtrees and their classes.

5.3 Hiding the adversarial attack

Our motivation for manipulating the neural net features was to illustrate how the sparse oblique trees are able to gain information about how the network works internally. However, such manipulations can also be seen as adversarial attacks at the feature level (which may or may not be practically feasible). Applying the attack is very simple, as it needs no optimization (which is the case with many pixel-level attacks).

We can also make the attack less obvious. In practice with our trees (which have sparse weight vectors), the above masks only require setting to 0 or 1, always in the same place, a small number of features ($\approx 10\text{--}40$) out of the total (hundreds or thousands), so they would be easily detectable to an observer of either the masked features or the softmax values at the output. We can easily randomize the above masks (and make them continuous rather than binary) so that they still work as intended but vary for each instance. First, the wildcard indices $*$ in the multiplicative masks can take any real value (positive, negative or zero). Second, the additive mask can take any positive values as long as they are small enough. Third, there typically is a subset of features which are not used in any node of the tree, so they can also take any real value.

Changing some additional features may of course have some unintended effects in the deep net. However, this can be reduced by applying the above randomization to a small, itself random number of features in the mask. Also, since the deep net features are the output of a ReLU, some of them are 0 to start with, so the mask has no effect there anyway.

Some of our manipulations are less detectable than others by their nature. Two classes that are siblings (such as ‘4’ and ‘9’ in the MNIST tree; Fig. 11) are likely more similar than if they are far apart in the tree, so misclassifying them (as one of our masks does) would not raise much suspicion. The attack described in this section is a white-box attack, which means the attacker has access to the model.

6 Experiments

We have evaluated our trees and masks thoroughly on two deep nets:

- VGG16 (Simonyan and Zisserman 2015) in a subset of 16 classes of ImageNet (Deng et al. 2009), for which we select the $F = 8192$ neurons from its last convolutional layer. Table 2 gives the network architecture.
- LeNet5 in MNIST on 10 digit classes (LeCun et al. 1998), for which we select the $F = 800$ neurons at layer conv2 as features. Table 3 gives the network architecture.

Table 1 Classes in our ImageNet subset and their id (for reference in other figures)

Label id	Class
0	Goldfish
1	Bald eagle
2	Goose
3	Killer whale
4	Siberian husky
5	White wolf
6	Tiger cat
7	Lion
8	Airliner
9	Container ship
A	Fire engine
B	School bus
C	Speedboat
D	Sports car
E	Warplane
F	Coral reef

For both of them, we can train trees that accurately mimic the deep net classifier **g**. The trees give remarkable insight in the relation of deep net features to classes and allow us to construct masks that indeed work as intended in the deep net for most instances. We describe this in detail next.

6.1 Results on VGG16 (subset of ImageNet dataset)

We selected a subset of 16 classes from the ImageNet object classification dataset (Deng et al. 2009); they are listed in Table 1. For each class we split the available images into 200 for test, 100 for validation and 1 000 for training (total 20 800 images). We used a VGG16 deep net (Simonyan and Zisserman 2015) with the architecture shown in Table 2, which takes as input color images of 64×64 pixels. We fine-tuned a pretrained VGG16 for our ImageNet subset of 16 classes. We train the network using Nesterov's accelerated gradient method for 20 epochs using minibatches of size 32, learning rate 0.02 and momentum rate 0.9. Our resulting VGG16 net achieves an error of 0.2% (training) and 6.79% (test). We select the $F = 8\,192$ neurons from VGG16's last convolutional layer as features on which to train the tree.

We trained sparse oblique trees on these features with the TAO algorithm, using our own Python implementation of TAO. We used as initial tree structure for TAO a complete tree of depth 6 (total 127 nodes), which we found sufficient to produce small but accurate trees in this case, and random initial values for the weights at the nodes. The decision nodes are hyperplanes and each leaf contains a single class label. We constructed a collection of trees over a range of the sparsity hyperparameter $\lambda \in [0, \infty)$ (the regularization path). We ran TAO for 40 iterations (when it approximately

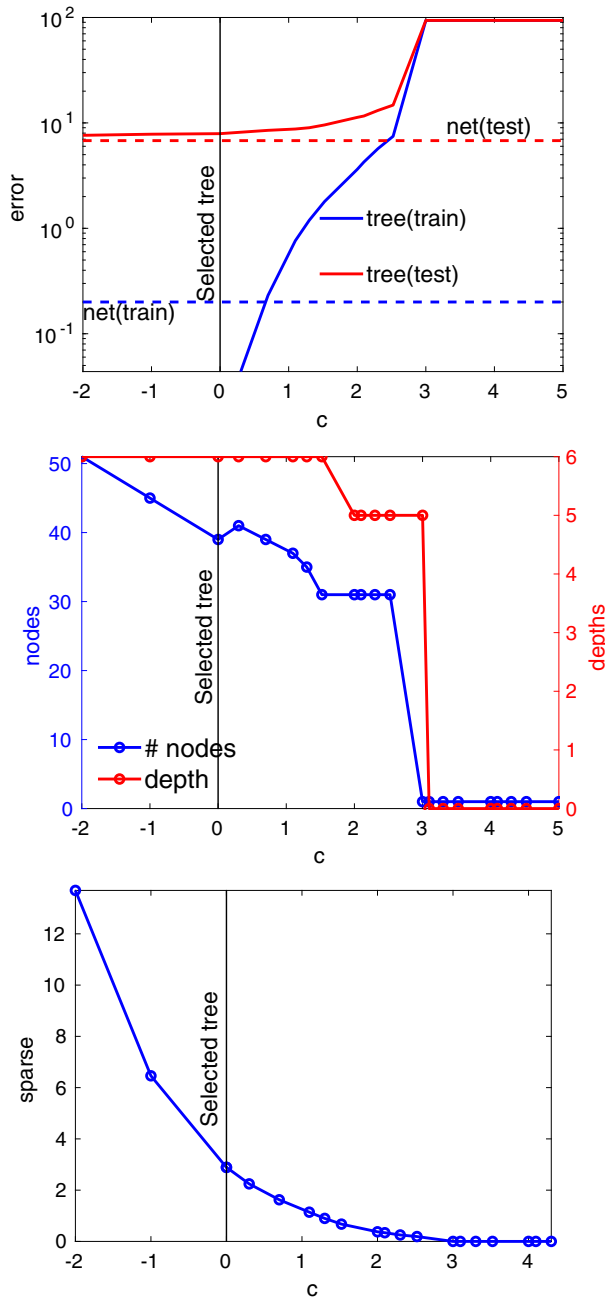


Fig. 4 Classification error (training and test) and number of nodes and of nonzero weights of the trees as a function of λ for VGG16. The vertical line indicates the tree we selected as mimic ($\lambda = 1$)

converged) to learn each tree. Figure 4 shows, for each tree, its error (training and test), size (depth and number of nodes) and number of nonzero weights as a function of the sparsity hyperparameter $\lambda \in [0, \infty)$.

The tree with the lowest error over the values of λ we considered occurred for $\lambda = 0.01$. It has depth 6 and 51 nodes, and an error of 0% (training) and 7.62% (test). It uses only 4423 features out of the total 8192. We did not use this tree, instead we selected as mimic the tree for $\lambda = 1$, which is quite smaller (depth 6 and only 39 nodes) but has nearly the same error (0% training, 7.90% test). It uses just 1366 features (17% of the total 8192). Its error is very close to that of VGG16, so we expect the tree to be a good mimic of the net. We normalize the final tree so each node weight vector has norm 1. This tree is shown in Fig. 5. We also discuss another tree that is slightly less accurate but which has exactly one leaf per class and is even more interpretable (Fig. 6). This tree ($\lambda = 33$) has an error of 1.79% (training) and 9.56% (test); it has 31 nodes and uses just 408 features (5% of the total 8192).

6.1.1 Inspecting the sparse oblique trees

Figure 4 shows that as we increase λ and therefore impose increasing sparsity on the tree (in terms of both tree size and number of nonzero weights in the decision nodes), the training error increases steadily but the test error remains about constant, so both curves approach, meet for some λ value and approximately coincide from that point on. We find this behavior in both MNIST and ImageNet. This provides opportunities to find a tree with pretty good accuracy but significantly sparse.

Figure 5 shows the tree we use as mimic. Its training and test errors are close to those of VGG16, so we expect it to be a good mimic, which indeed happens (see masks later). The top of the figure shows the class histogram at each node, i.e., the distribution of classes on the subset of training instances that a node receives. These histograms show how the tree hierarchically splits classes very crisply; indeed, it has only 20 leaves for 16 classes. In the bottom of the figure, the weight vector at each decision node shows that very few features are used at each node; indeed, 83% of the features are not used at any node, so their values are irrelevant for classification in the tree. This also holds nearly perfectly for the deep net, that is, the feature selection insights obtained for the tree transfer to the neural net. It suggests that some of the features and hence neurons and weights of the net are practically redundant, or perhaps that they code for properties that are useful for only a few specific instances. This is not surprising if one notes that deep nets (at least, as presently designed) seem to be vastly overparameterized and can be significantly compressed by pruning weights and neurons (Carreira-Perpiñán and Idelbayev 2018).

Figure 6 shows a very interesting tree, obtained for a larger λ value so that there is exactly one leaf per class (the smallest number of leaves possible unless we ignore classes). This tree has very few nonzero weights yet its test error is reasonable, so it probably extracts features that robustly classify most images. Also, its structure remains unchanged for a wide range of λ . *Inspecting it shows an intuitive hierarchy of classes that seem primarily related to the background or surroundings of the main object in the image.* Its leftmost subtree {warplane, airliner, school bus, fire engine, sports car} consists of man-made objects often found on roads. However, {container

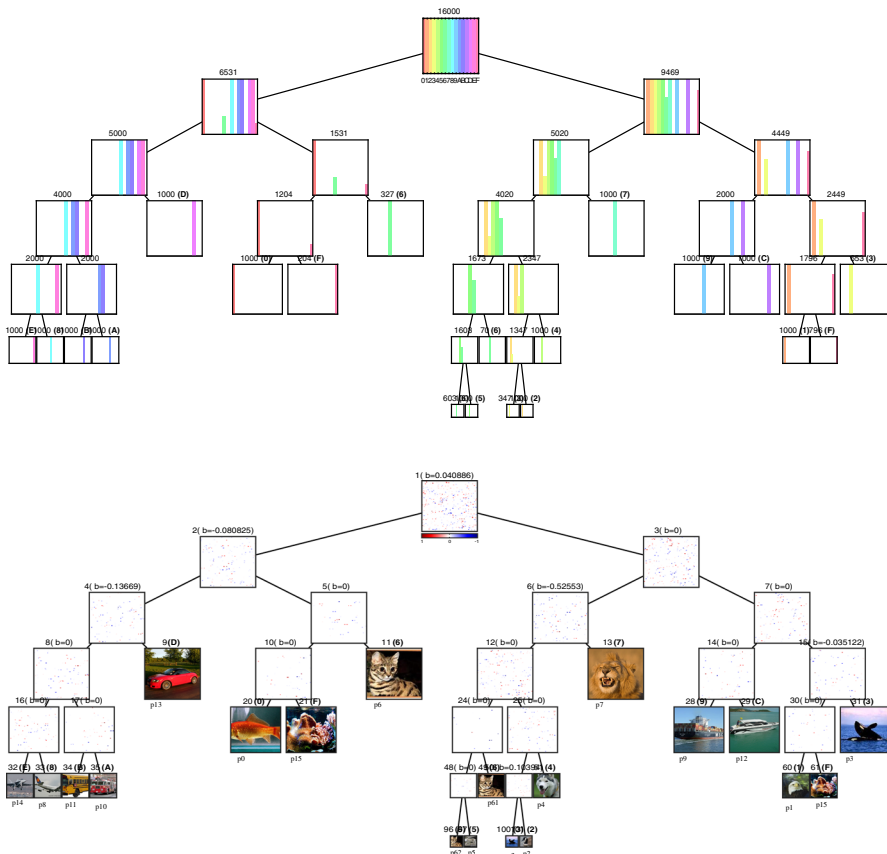


Fig. 5 Tree selected as mimic for VGG16 features ($\lambda = 1$), with a training error of 0% and a test error of 7.90%. *Top*: class histograms; we show the number of training instances reaching the node and, for leaves, their label. *Bottom*: weight vector at each decision node and an image from their class at each leaf; we show the node index, bias (always zero) and, for leaves, their label. We plot the weight vector, of dimension 8192, as a 91×91 square (the last pixels are unused), with features in the original order in VGG16 (which is determined during training and arbitrary, hence the random aspect of the images), and colored according to their sign and magnitude (positive, negative and zero values are blue, red and white, respectively). You may need to zoom in the plot

ship, speedboat} (man-made objects found on the sea) appears in the rightmost subtree, together with {killer whale, bald eagle, coral reef}, all of which are also typically found on the sea or on the air. Yet {goldfish} appears in a single subtree quite separate from all other classes: indeed, this fish is found on fishbowls (not the sea) in the training images. A subtree in the middle contains animals in land natural environments (forest, snow, grass, etc.): {tiger cat, white wolf, goose, Siberian husky, lion}. And so on. This is consistent with previous works that have found that, in some specific cases, the reason why a deep net classifies an object as a certain class is caused by the background or more generally by some confounding variables (Ribeiro et al. 2016; Zech et al. 2018). It points to a possible vulnerability of the net, in that it may badly

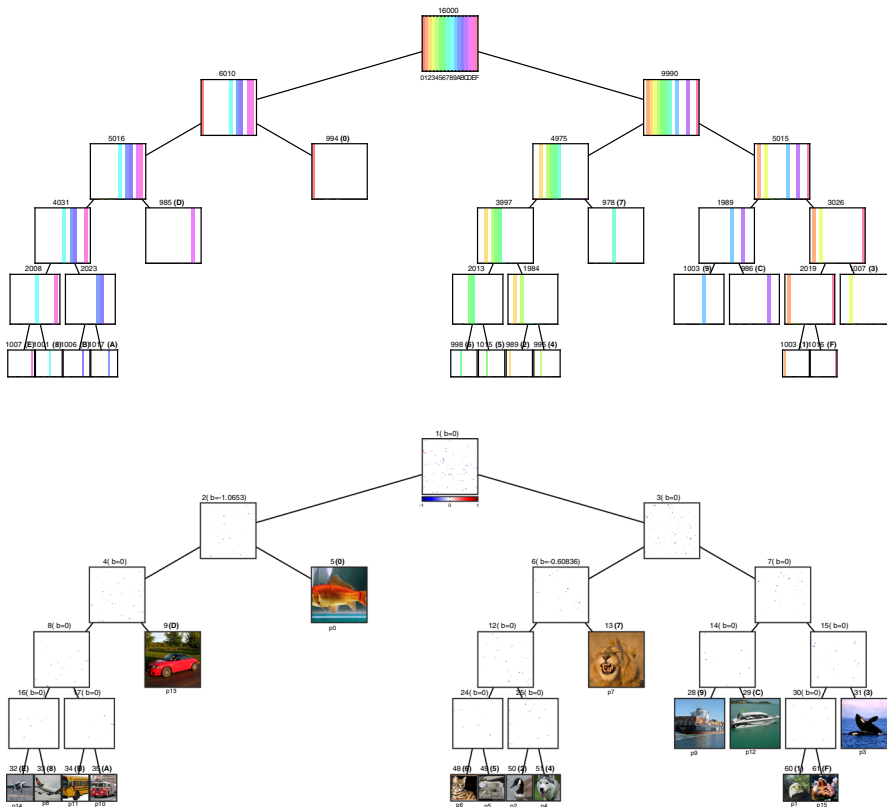


Fig. 6 Like Fig. 5 but for $\lambda = 33$. This tree has exactly one leaf per class (total 16 classes), and a slightly higher error (training error 1.79%, test error 9.56%)

misclassify an object that happens to appear in an unusual background (say, a bald eagle standing on a road).

6.1.2 Manipulating the deep net features via masks

We derive masks using the mimic tree ($\lambda = 1$). Figures 7, 8 show confusion matrices, which are self-explanatory, over all instances (test and training, respectively) regarding the deep net, the tree and the masked deep net. Generally, the masks affect the deep net classification in the same way as the tree. This is to be expected since the tree has a very similar error and confusion matrix as the net, but it is still surprising in how well it works in most cases. This also indicates that certain neurons of the deep net (those critically involved in the masks) play a well-defined role in the classification. The number of features that a mask critically needs to perform its job is very small, around 200 (out of 8 192); for MNIST it is much smaller, around 40.

We emphasize two things. First, these confusion matrices are constructed using *all instances (training or test)*. Hence, our conclusions are robust and global, unlike other works that either work locally on a single instance by design, or work globally but

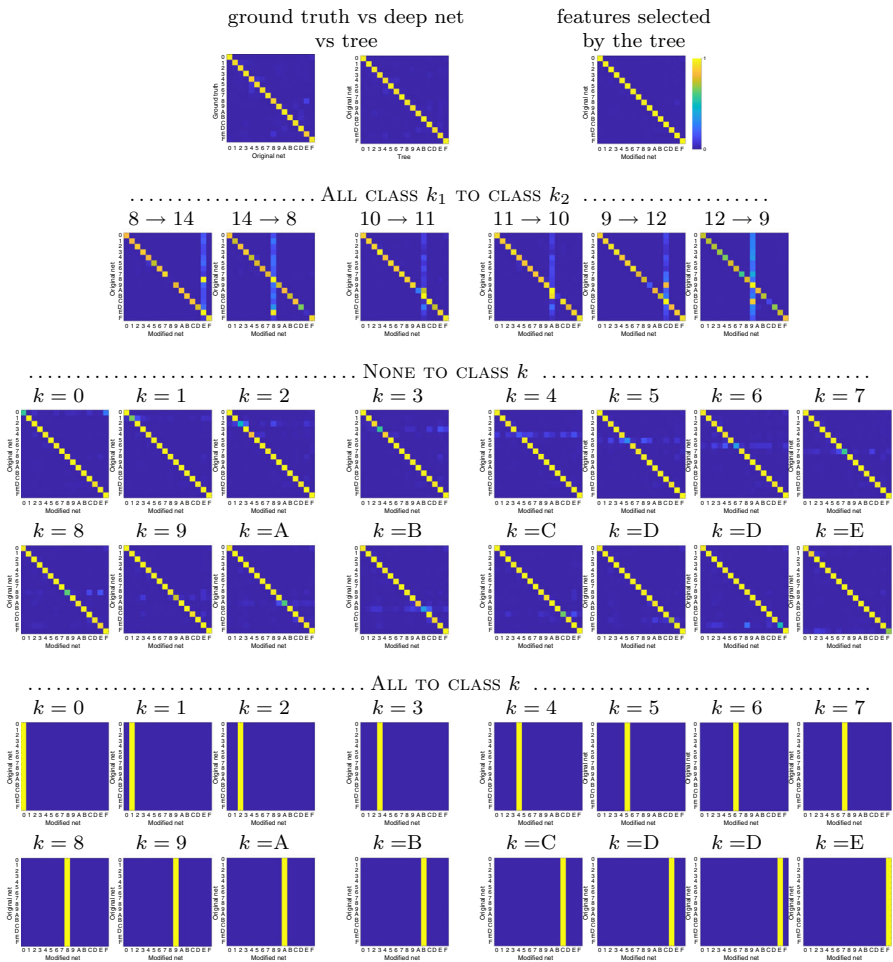


Fig. 7 Confusion matrices for VGG (test set). *First row left*: ground-truth vs deep net, and deep net vs tree. *First row right*: deep net vs deep net with only the features selected by the tree. *Second row*: ALL CLASS k_1 TO CLASS k_2 (selected examples). *Third and fourth row*: NONE TO CLASS k . *Fifth and sixth row*: ALL TO CLASS k

show good results on a handful of instances only. By reporting the results aggregated over all instances, we demonstrate the robustness of our masks. Second, *what we show is the result of applying the original VGG16 deep net to the masked features*, not of applying the tree to the masked features (which would work perfectly by construction). This shows that the masks constructed using the tree mimic transfer almost perfectly to the deep net.

Let us analyze the different panels of the figure in more detail:

- The two confusion matrices “ground truth vs deep net vs tree” (which look like a diagonal yellow line) show that the deep net and the tree predictions are almost

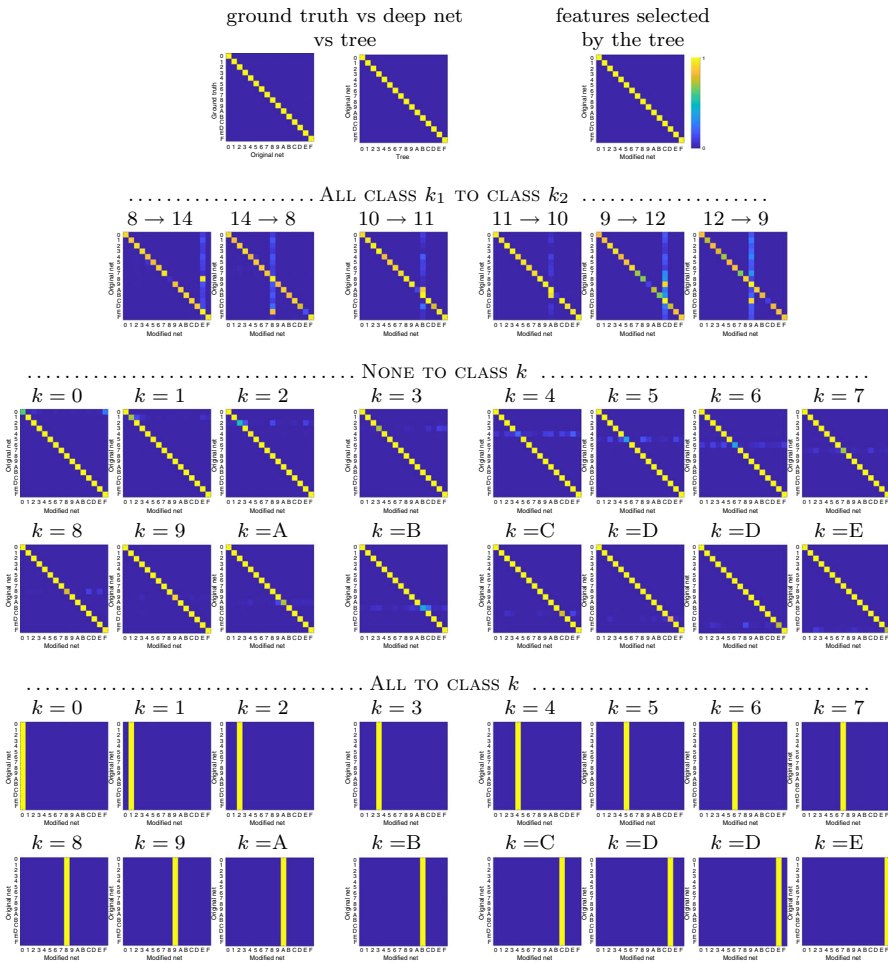


Fig. 8 Like Fig. 7 but for the training set

identical to each other and to the ground truth, indicating that the tree is a good mimic (blue is 0 and yellow 1).

- The confusion matrix “features selected by the tree” (which looks like a diagonal yellow line) shows that if we mask out all features in VGG16 except those selected by the tree, the classification ability remains the same, indicating that the tree is able to detect a subset of features that are enough for good classification.
- The 6 confusion matrices “ALL CLASS k_1 TO CLASS k_2 ” demonstrate (in selected combinations of k_1 and k_2) that this mask works quite reliably. Ideally, entry (k_1, k_1) should become 0 and entry (k_2, k_1) should become 1, with the rest of the entries remaining unchanged. We often observe a bluish vertical bar at k_2 , indicating that a small proportion of the k_1 instances are classified as a class different from k_2 .

- The 16 confusion matrices “NONE TO CLASS k ” (for $k \in \{0, 1, \dots, F\}$) should ideally make the entry at (k, k) equal to zero (this entry is originally equal to (almost) 1) and distribute its mass over any other classes (entries (k, k') with $k \neq k'$), with the rest of the entries remaining unchanged. This succeeds better in some classes than in others, but generally works well.
- The 16 confusion matrices “ALL TO CLASS k ” (for $k \in \{0, 1, \dots, F\}$) should ideally look like a vertical yellow line at k , and indeed they do in all cases.

All of the above is true for both training and test instances, although the masks work slightly better for the training instances.

6.1.3 Illustration of the masks with an actual image

Figure 9 illustrates the mask behavior in an image not in the dataset. The middle column histograms show the deep net features (grouped by class). In each row, the top histogram shows the feature values, and the bottom histogram shows the number of features selected for each class. Next, we show how masking the features drastically alters in a controlled way the softmax output. In row 2, when we apply the ALL TO CLASS “SIBERIAN HUSKY” mask, the network now classifies the image as “siberian husky”. Similarly, in row 6, when we apply the ALL TO CLASS “BALD EAGLE” mask, the network now classifies the original image as “bald eagle” with large confidence, compared to row 1, where without the mask the softmax value for “bald eagle” is close to zero. We also show how the mask correlates with superpixels (perceptual groups of pixels obtained by oversegmentation) in the image, either manually cropped (row 3) or optimized to invert the desired deep net features (row 4).

To obtain results like those above, the general procedure is as follows. Firstly (in an offline phase), we train the tree mimic and construct a subset of features $\mathcal{S}_k$ for each class k , using the ALL TO CLASS k mask. This defines a score for an input image $\mathbf{x}$ as $s_k(\mathbf{x}) = \sum_{i \in \mathcal{S}_k} F_i(\mathbf{x})$, where $F_i(\mathbf{x})$ is the feature i computed by the deep neural net for $\mathbf{x}$. We can then discard the tree and the classifier part of the deep net. All we need is the feature-extraction part of the deep net and the class sets $\mathcal{S}_1, \dots, \mathcal{S}_K$.

Then (in an online phase), given an input image and a target class k , we split the image into superpixels, compute the score for each superpixel, and report the superpixels with lowest score (most salient). We use the quick-shift segmentation algorithm (Vedaldi and Soatto 2008) implemented in the `skimage`⁴ library to generate superpixels. We set all parameters to their default values except the color-space and image-space proximity ratio, and the cut-off point for data distances, which we set to 0.2 (more weight to image-space proximity) and 200 (fewer clusters), respectively.

6.1.4 Control experiments

We run some experiments to test the robustness of our findings (see details in Appendix 2):

1. We trained trees with TAO using 5 different random initial trees and verified we can achieve very similar results to those we report (masks, etc.). Note that, for a

⁴ <https://scikit-image.org/docs/0.15.x/>

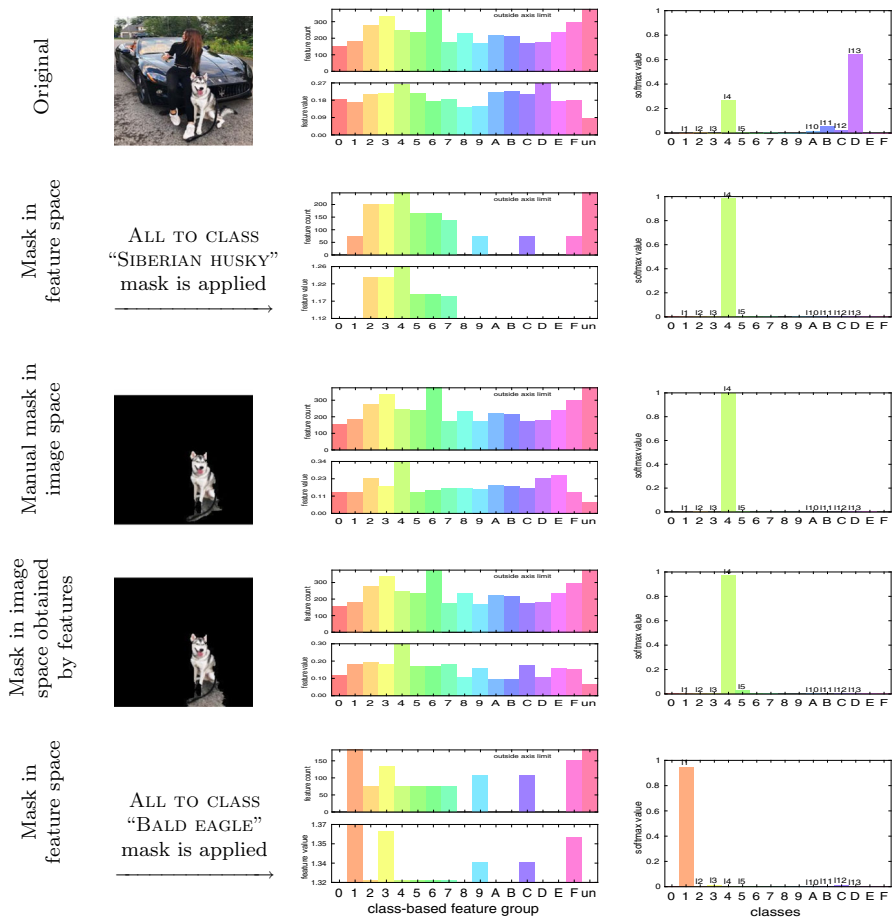


Fig. 9 Illustration of masks for a particular image (VGG16 network on ImageNet subset). Column 1 shows the image masks (when available). Column 2 summarizes the 8 192 feature values as two histograms: on the upper panel, the number of features in each class group (listed in the X axis as 0–F, where “*” means features not used by the tree); on the lower panels, the average feature value (neuron activation) per class group. Column 3 shows the histogram of corresponding softmax values. Row 1 shows the original image. Row 2 shows a mask in feature space to classify it as “Siberian husky”. Row 3 shows a mask manually cropped in the image, whose features resemble those of row 2. Row 4 shows a mask in image space obtained by finding the top-3 superpixels whose features most resemble those of the masked features of row 2. Row 5 shows a mask in feature space to classify the image as “bald eagle”

- given neural network and dataset, it is conceivable that we could learn very different trees (even resulting in different masks) because of local optima in the tree training, correlation or redundancy of features in the neural network, etc. But regardless of that, because we evaluate the masks not just in the tree but in the original neural network, we can claim that the chosen masks work as described earlier.
- We trained a CART tree (Breiman et al. 1984; Therneau et al. 2019; Pedregosa et al. 2011) on the VGG16 features. This created a huge axis-aligned tree with error of 1.97% (training) and 21.2% (test), depth 54, 1 381 nodes and using 619 features.

The large test error means the tree is inadequate as a mimic of VGG16, and the tree size would make it useless for explanation purposes anyway. This demonstrates the inability of CART to learn accurate trees for complex, high-dimensional data.

3. We trained a TAO tree on a rotated version of the VGG16 features, i.e., multiplying the feature vector by an orthogonal matrix. This has the effect of mixing all the features in an invertible way. Without sparsity ($\lambda = 0$), the TAO tree achieves identical error to the unrotated features' tree (since the oblique nodes can absorb any linear transformation). However, when increasing λ in order to force the tree to use few features, the test error jumps to 11.3%, much higher than with the unrotated features. This shows that the features learnt by VGG are special in that they seem to operate in small groups associated with classes, rather than all or most features participating in each class.

6.2 Results on LeNet5 (MNIST dataset)

We trained sparse oblique trees on features obtained by the LeNet5 neural net architecture for MNIST. The results agree qualitatively with those for VGG16: we are able to obtain trees with nearly the same error as the original net, hence good mimics; the masks we construct work as desired in nearly all the training and test instances; and the tree is highly interpretable.

Specifically, we train a LeNet5 net on the 60 000 training images for MNIST, of 28×28 pixels (10 handwritten digit classes), and report test errors on the 10 000 test instances (LeCun et al. 1998). The LeNet5 architecture is in Table 3. We train the network using Nesterov's accelerated gradient method for 100 epochs using minibatches of size 512, learning rate 0.02 (updated every 20 epochs with a factor of 0.99^2) and momentum rate 0.9. The training error is 0.00545% and the test error is 0.61%. We select the $F = 800$ neurons at layer conv2 as features on which to train the tree.

We used TAO with an initial tree structure of depth 5 (total 63 nodes) and random initial values for the weights at the nodes. We constructed a collection of trees over a range of the sparsity hyperparameter $\lambda \in [0, \infty)$ (the regularization path), running TAO for 40 iterations (when it approximately converged) to learn each tree. Figure 10 shows, as a function of λ , the error (training and test), number of nodes and depth of each tree, and number of nonzero weights in each tree (total over all its nodes). As with the VGG16 trees, as we increase λ and therefore impose increasing sparsity on the tree (in terms of both tree size and number of nonzero weights in the decision nodes), the training error increases steadily but the test error remains about constant, so both curves approach, meet for some λ value and approximately coincide from that point on. This provides opportunities to find a tree with pretty good accuracy but significantly sparse. As we increase λ , the tree size decreases (number of nodes and depth) and the number of nonzero weights decreases.

We selected as mimic the tree for $\lambda = 20$, with depth 5 and only 27 nodes. It has an error of 1.28% (training) and 1.67% (test), which is very close to that of LeNet5, so we expect the tree to be a good mimic of the net. The best classifier tree (for $\lambda = 5$) had a depth of 5 and 27 nodes, and an error of 0.59% (training) and 1.46% (test). It is

possible to reduce this error even more by using a larger tree, but the one we obtained is good enough as a mimic and to obtain masks.

Figure 11 shows the tree selected as mimic. The class histograms at each node show how the tree hierarchically splits classes very crisply; indeed, it has only 14 leaves for 10 classes (only digit classes ‘2’, ‘6’, ‘7’, ‘9’ appear with 2 leaves each). The blurry average image at each leaf shows significant shape variability, indicating the features have successfully learned to ignore such within-class variability. The weight vector at each decision node shows that very few features are used at each node; 295 features (37% of the total 800) are not used at any node, so their values are irrelevant for classification in the tree. This also holds nearly perfectly for the deep net.

Figures 12, 13 show the confusion matrices for our different masks, which work nearly perfectly in the training instances and only slightly less so in the test ones. The number of features we need to mask out in each case is very small, around 40 (out of 800 features). Some masks work more reliably than others. Classifying all instances as class k works surprisingly well no matter the choice of k . Misclassifying class k_1 as k_2 (where k_1 must have a single leaf which is a sibling of k_2) works also well, although a few instances from other classes are sometimes classified as k_2 . Not classifying any instance as class k works also well but fails with some instances, which remain as class k .

It is interesting to compare our tree with a TAO tree trained directly on the pixel values (hence not associated with any deep net), such as that in (Fig. 1 Carreira-Perpiñán and Tavallali (2018)). The tree trained on LeNet5 features is much smaller, sparser and accurate. The best test error for a tree on pixels (panel 2 of (Fig. 1 Carreira-Perpiñán and Tavallali (2018))) is 5.69% for a tree of depth 8 and 75 nodes (in our own experiments we can get this down to around 5%). This error is remarkably low for a tree but, compared to our tree, the error is much bigger, and the tree is quite larger and messier. A smaller, more interpretable tree on pixel values shown in panel 3 of (Fig. 1 Carreira-Perpiñán and Tavallali (2018)) achieves an error of around 10% (training or test) with depth 7 and 33 nodes. Since the tree operates on the raw pixels, inspection of the weight vectors is very informative in identifying “strokes” that characterize the difference between a digit 4 and a digit 9, for example. As to applying our masks at the pixel level, this is far less successful if we want guarantees for (nearly) all instances; however, it may be possible to make this work for a specific image.

7 Discussion and limitations of our work

Decision trees are a good choice of interpretable classifier because they handle multiple classes naturally, do feature selection automatically, have a hierarchical structure that promotes an increasing specialization from the root towards the leaves, and can be inspected. Sparse oblique trees improve this by using few features at each node, so they explicitly show the influence of groups of features on classes. This makes it possible to find important subsets of features efficiently among thousands of possible features. For such trees to be useful, it is critical to be able to train them to high accuracy so they can mimic (part of) a deep net, which the TAO algorithm does.

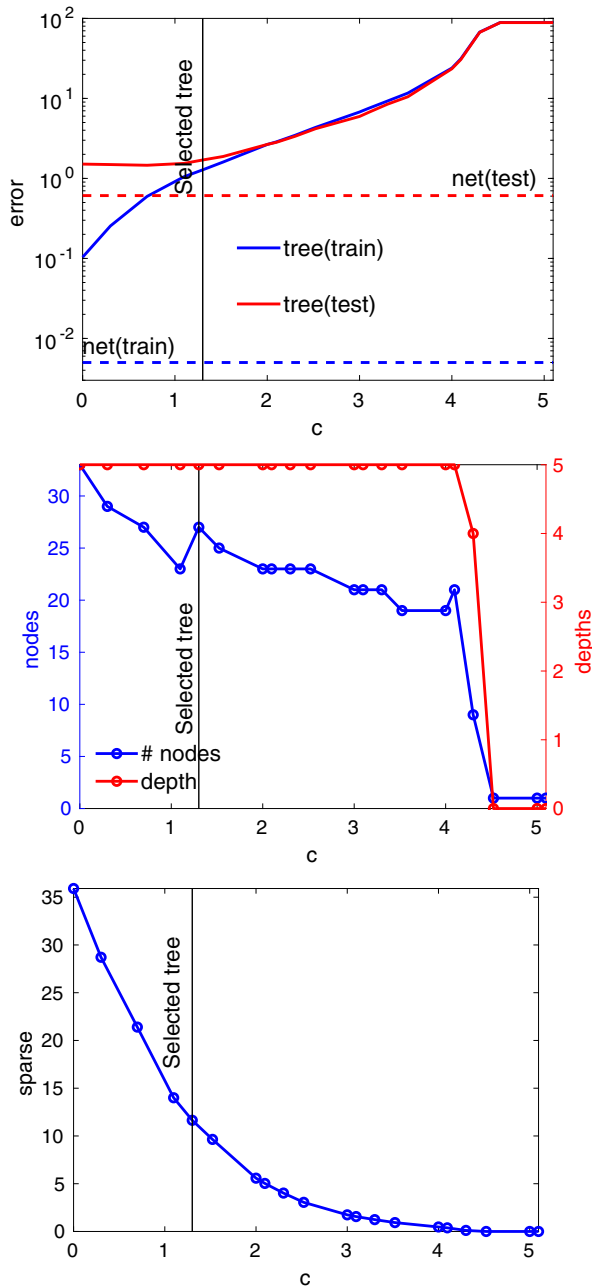


Fig. 10 Classification error (training and test) and number of nodes and of nonzero weights of the trees as a function of λ for LeNet5. The vertical line indicates the tree we selected as mimic ($\lambda = 20$). Compare this with Fig. 4

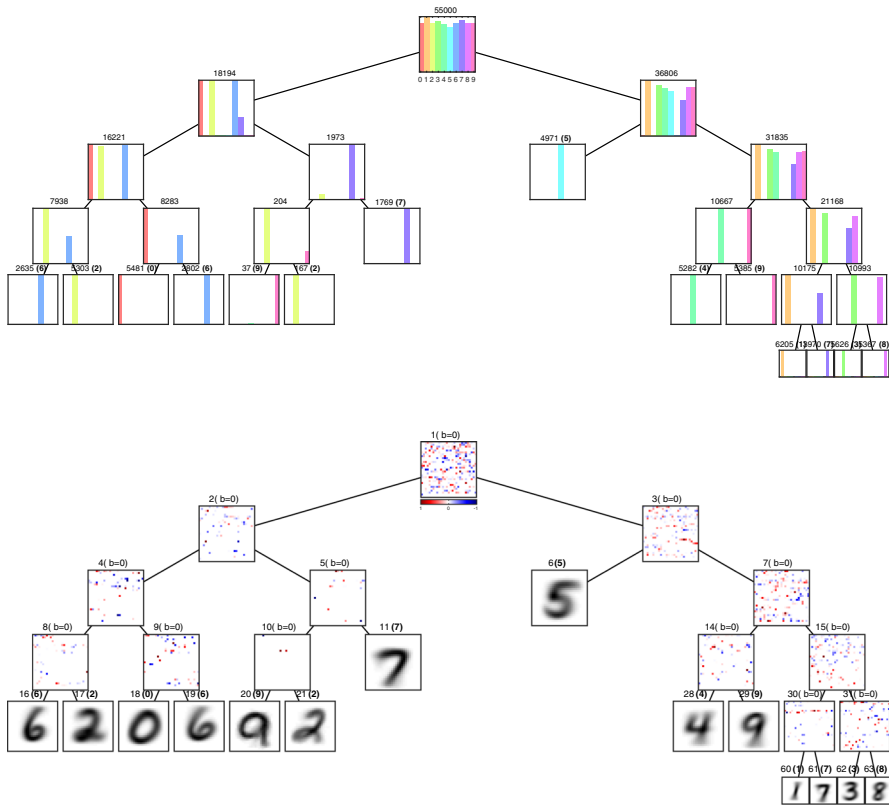


Fig. 11 Tree selected as mimic for LeNet5 features ($\lambda = 20$). *Top*: class histograms; we show the number of training instances reaching the node and, for leaves, their label. *Bottom*: weight vector at each decision node and average of training instances at each leaf; we show the node index, bias (always zero) and, for leaves, their label. We plot the weight vector, of dimension 800, as a 29×29 square (the last pixels are unused), with features in the original order in LeNet5 (which is determined during training and arbitrary, hence the random aspect of the images), and colored according to their sign and magnitude (positive, negative and zero values are blue, red and white, respectively). You may need to zoom in the plot

As shown by our masks, deep net features participate in a coordinated way in predicting a class, where small groups of specific features encode information specific for some decisions (rather than, say, all features participating in all classes for all instances). That we find features specialized for classes is not that surprising—some features must provide information for some classes, after all. What is surprising is the small size of these groups of features. This is probably partly due to the neural network being heavily overparameterized. What do these groups of neural net features represent anyway? Unfortunately, in spite of the high activity in this area (as summarized in Sect. 2), at present the research community does not have a systematic understanding of what “concepts” these neural net features may be encoding; elucidating this remains an open research problem.

Our findings are remarkably similar to recent findings in visual neuroscience (Marshall et al. 2019; Carrillo-Reid et al. 2019) that show that very small groups of neu-

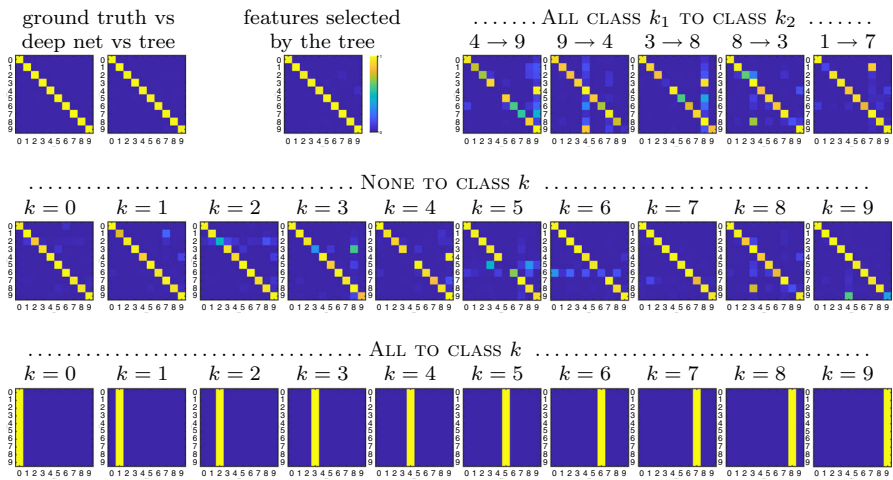


Fig. 12 Confusion matrices for LeNet5 (test set). *Top left*: ground-truth vs deep net, and deep net vs tree. *Top middle*: deep net vs deep net with only the features selected by the tree. *Top right*: ALL CLASS k_1 TO CLASS k_2 (selected examples). *Middle*: NONE TO CLASS k . *Bottom*: ALL TO CLASS k

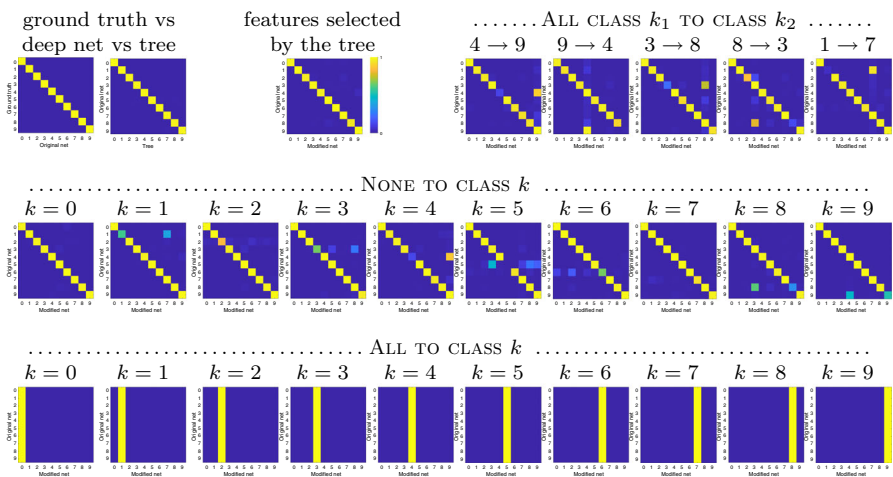


Fig. 13 Like Fig. 12 but for the training set

rons (around 20) in mouse primary visual cortex seem to code for specific properties or behaviors. In fact, removing all visual input to the mouse and directly stimulating those neurons triggers the same behaviors—analogously to what our masks do.

Recent work (Kuhn and Kacker 2019) tries to address the problem of identifying a group of n features (out of m total features) that are related with a specific class. They consider explainability of a classifier as a problem of fault location in combinatorial testing. Specifically, they seek to identify combinations of features that are present in members of a given class but absent or rare in non-members. However, finding a group of n features out of m existing features involves m -choose- n combinations. This does

not scale to neural networks such as LeNet5 or VGG16, which use a large number of features.

Our results apply to features extracted by a deep net with specific weights. Since deep nets are typically overparameterized and have local optima, it is possible to obtain numerically very different weights depending on the initialization and optimization protocol. We have not explored how this may affect the resulting features, tree and masks. We did observe that our results seem robust to the initialization of the tree itself.

One fair criticism of mimic-based interpretations is that, in learning a mimic, what we are interpreting is the mimic, not the original model (here, the neural net). For example, if the mimic strongly uses feature i to classify into class k (say), then it is tempting to conclude that the original model also does that. This is risky, especially if the mimic is not very accurate with respect to the neural net. However, in our experiments we transferred the insights obtained through the tree mimic to the original neural net (e.g. in masking features) and confirmed that they still hold there for most of the training and test instances.

8 Conclusion

Our paper demonstrates the use of sparse oblique decision trees as a powerful “microscope” to investigate the behavior of deep nets, by learning interpretable yet accurate trees that mimic the classifier part of a deep net. The tree takes as input the neural net “features” produced by the neural net for a given input instance (that is, the neural net activations at an internal layer). The tree then predicts the corresponding class for those features, emulating the classification behavior of the neural net with very high accuracy, at least in the neural nets we considered. Using oblique trees trained with the TAO algorithm is critical for this to succeed.

The resulting tree gives insights about the relation between neurons and classes, such as what groups of neurons are involved in predicting what classes. It also enables the design of simple manipulations of the neuron activations that can, for any training or test instance, change the class predicted in various, controllable ways (thus making adversarial attacks possible at the level of the deep net features).

This approach to interpreting or manipulating features applies to other types of deep nets and data, such as audio or language. It may also prove helpful in other areas where deep nets are being applied, such as in biology, where we may be able to relate neurons to genes or diseases, and observe the effect of “knocking out” such genes, which is essentially what our proposed masks do.

Acknowledgements Work partially supported by NSF award IIS–2007147.

Appendix A: Neural network architectures

Tables 2 and 3 describe the architectures for our VGG16 network and the LeNet5 network.

Table 2 Architecture of our modified VGG16 neural net for our ImageNet subset (64 × 64 image size)

Layer	Connectivity	Layer	Connectivity
Input			
1	64 × 64 × 3 Image convolutional, 64 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU	11	convolutional, 512 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU
2	convolutional, 64 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU	12	convolutional, 512 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU
3	max pool, 2 × 2 window (stride=2)	13	convolutional, 512 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU
4	convolutional, 128 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU	14	max pool, 2 × 2 window (stride=2)
5	convolutional, 128 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU	15	convolutional, 512 3 × 3 filters (stride=1,padding = 1) followed by BatchNormalization → ReLU

Table 2 continued

Layer	Connectivity	Layer	Connectivity
6	max pool, 2 × 2 window (stride=2)	16	convolutional, 512 3 × 3 filters (stride=1, padding = 1) followed by BatchNormalization → ReLU
7	convolutional, 256 3 × 3 filters (stride=1, padding = 1) followed by BatchNormalization → ReLU	17	convolutional, 512 3 × 3 filters (stride=1, padding = 1) followed by BatchNormalization → ReLU
8	convolutional, 256 3 × 3 filters (stride=1, padding = 1) followed by BatchNormalization → ReLU	18	Dense Layer 4096 neuron followed by ReLU → Dropout (p=0.6)
9	convolutional, 256 3 × 3 filters (stride=1, padding = 1) followed by BatchNormalization → ReLU	19	Dense Layer 4096 neuron followed by ReLU → Dropout (p=0.6)
10	max pool, 2 × 2 window (stride=2)	20	Dense Layer 16 neuron
		21	softmax

Table 3 LeNet5 architecture

Block name	Block description
conv1	convolution with kernel size 5×5 and 20 channels ReLU maxpooling with kernel size 2×2
conv2	convolution with kernel size 5×5 and 50 channels ReLU maxpooling with kernel size 2×2
fc1	Fully connected layer with 500 hidden units ReLU
dropout	$p=0.5$
fc2	Fully connected layer with 10 hidden units

Appendix B: Control experiments

CART trees are unsuitable as mimics for the deep net

Sparse oblique trees trained with TAO have been shown (Carreira-Perpiñán and Tavalali 2018) to outperform traditional tree learning algorithms by a large margin, in particular axis-aligned trees trained with CART (Breiman et al. 1984; Therneau et al. 2019; Pedregosa et al. 2011). Still, we tried to construct a mimic by using an axis-aligned tree trained with CART. In an axis-aligned tree, each decision node tests a single feature (rather than a linear combination) in order to send an instance down its left or right child. We used the CART implementation of scikit-learn (Pedregosa et al. 2011). As is customary with CART, we first allow the tree to grow in full and then apply cost-complexity pruning, choosing the best pruning hyperparameter by cross-validation. We learned the CART tree on the same dataset of VGG16 features as the TAO tree. Table 4 shows statistics of the resulting tree and Fig. 14 the tree itself. It is obvious that the tree is both grossly inaccurate in test error (21.2%) and huge in size (depth 54 and 1381 nodes). This makes it unsuitable as a mimic for VGG16 and practically impossible to interpret or to construct masks.

Rotated deep net features do not admit sparsity well

Our finding that a very small subset of deep net features are sufficient to control classification for a given class is surprising, because it is not strictly necessary for this to happen. That is, each class could be dependent on the collaborative values of all (or nearly all) features. To verify this, we manufactured a version of the features that contains all the information in the original features, but mixes them in a dense way as a linear combination. Specifically, we multiplied the original features $\mathbf{f}$ by a dense, invertible matrix $\mathbf{Q}$, to obtain transformed features $\bar{\mathbf{f}} = \mathbf{Q}\mathbf{f}$. Clearly, both the fully-connected layers of VGG16 or an oblique tree can absorb this transformation, namely by using a new matrix $\bar{\mathbf{W}} = \mathbf{W}\mathbf{Q}^{-1}$ in the first fully-connected layer or by using a new weight vector $\bar{\mathbf{w}}_i = \mathbf{Q}^{-T}\mathbf{w}_i$ in each decision node i of the tree, respectively (since then we have $\bar{\mathbf{W}}\mathbf{f} = \mathbf{W}\mathbf{f}$ and $\bar{\mathbf{w}}_i^T \bar{\mathbf{f}} = \mathbf{w}_i^T \mathbf{f}$, where $\mathbf{W}$ and $\mathbf{w}_i$ were the original matrix or weight vector). However, this need not be true anymore if we force the weight vector $\bar{\mathbf{w}}_i$ to be sparse by using a large enough λ value. (The same would be true for the deep net if pruning weights in the fully-connected layers.) Hence, we expected that training

Table 4 Training a tree on VGG 16 features: axis-aligned tree with CART, sparse oblique tree with TAO

	CART after pruning	Before pruning	TAO $\lambda = 0.01$	$\lambda = 1$	$\lambda = 33$
Training error (%)	1.97	0.00	0.00	0.00	1.79
Test error (%)	21.24	21.47	7.63	7.91	9.56
Features used (out of 8 192)	619	795	4423	1366	408
Depth	54	57	6	6	5
Number of nodes	1381	1785	51	39	31

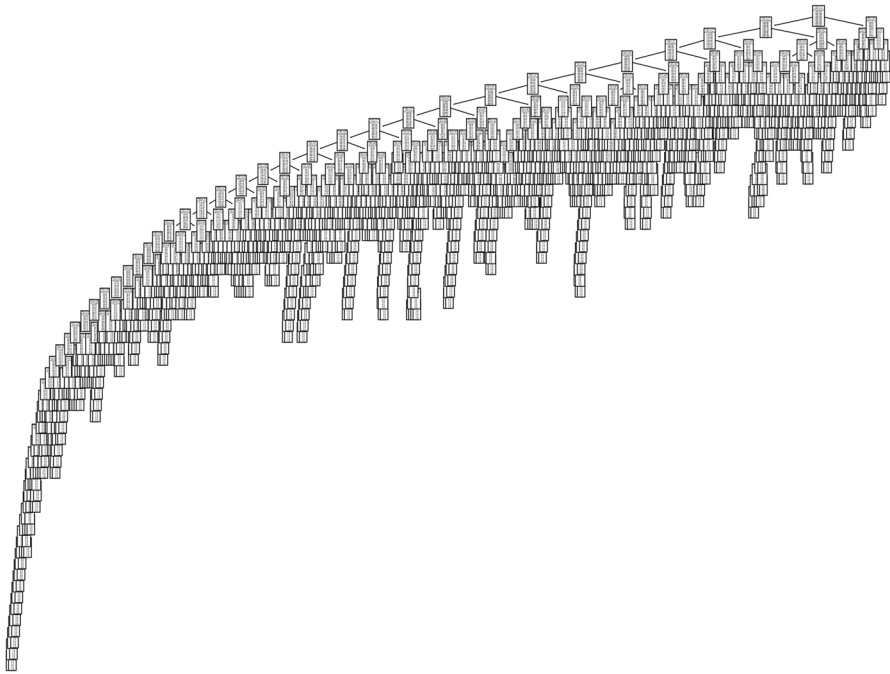


Fig. 14 CART axis-aligned tree trained on VGG16 features

a sparse oblique tree on the rotated VGG16 features would fail to produce a tree as sparse as before but with low test error. Indeed this is what happened, as described next.

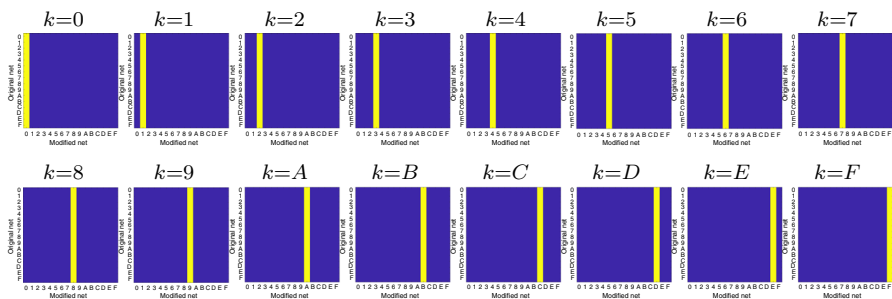
To generate a rotated version of the VGG16 features, we used a dense orthogonal matrix as $\mathbf{Q}$ matrix⁵. This has the desired effect of mixing all the features in an invertible way. Without sparsity ($\lambda = 0$), the TAO tree achieves identical error to the unrotated features' tree (as predicted theoretically). As we increase λ in order to achieve sparsity, the behavior of the tree is very different to that of Fig. 4, where the tree size, number of nonzeros and training/test error change continuously with λ . Instead, increasing the value of λ over a wide range (including the values used for our original trees) results in no sparsity at all. But once we reach $\lambda = 45$, the tree changes drastically: it becomes quite sparse but its test error jumps to 11.3%, much higher than with the unrotated features.

This shows that the features learnt by VGG are special in that they seem to operate in small groups associated with classes, rather than all or most features participating in each class. Since VGG16 could learn mixed rather than sparse features, the reason must be not in the architecture of VGG16 but in the training algorithm and/or objective function.

⁵ This satisfies $\mathbf{Q}^{-1} = \mathbf{Q}^T$ and is easier to handle. We use the default matrix in Matlab's `gallery('orthog', n, k)` function, which is an $n \times n$ matrix with entries defined as $q_{ij} = \sqrt{\frac{2}{n+1}} \sin\left(\frac{\pi ij}{n+1}\right)$.

Table 5 TAO sparse oblique trees trained with the original VGG features and the rotated VGG features

Original features			
λ	# features selected by the tree out of 8192	Training error (%)	Test error (%)
0.01	4423	0.00	7.63
1	1366	0.00	7.91
33	408	1.79	9.56
Rotated features			
λ	# features selected by the tree out of 8192	Training error(%)	Test error(%)
1	8192	0.00	7.91
45	335	1.91	11.34

**Fig. 15** Confusion matrices for VGG (test set) using the ALL TO CLASS k mask created with our sparse oblique trees

Linear classifier and CART trees are not suitable for explanation

Instead of sparse oblique trees, one could use other interpretable models to try to understand the relationship between the classes and the neurons. We show results (on the VGG16 features) using two other models that are widely considered as interpretable: a linear classifier and an axis-aligned tree (trained using CART). As shown next, they do not work nearly as well.

Softmax linear classifier First, we train a model without ℓ_1 regularization (so the weights are not sparse). This gives a reasonably good mimic, with a training error of 0.52% and a test error of 8.04%. However, all features participate in all classes, which makes the model difficult to interpret. Next, we train a model with ℓ_1 regularization and tune the latter to achieve a similar sparsity (around 83%) as our tree (Fig. 5). The linear classifier achieves a training error of 1.12% and a test error of 9.84%, which is worse but not too far from our tree (0% train and 7.63% test error). However, we cannot find class-specific neurons in this linear classifier. This is evident from Fig. 16, where we show the results of the ALL TO CLASS k mask created by class-specific neurons from the linear model. We can see that the mask fails for every class, meaning the linear model cannot identify the class-specific neurons.

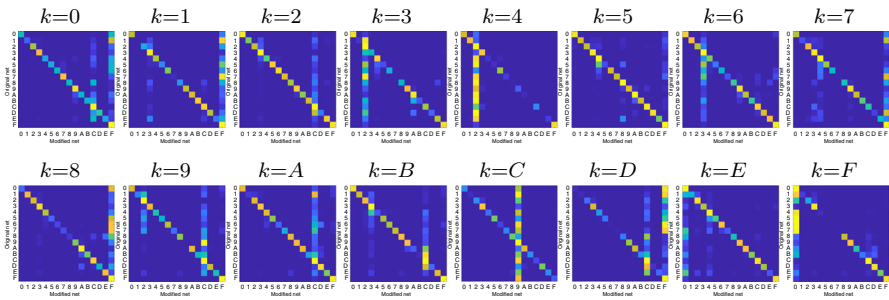


Fig. 16 Like Fig. 15 but using a sparse linear classifier

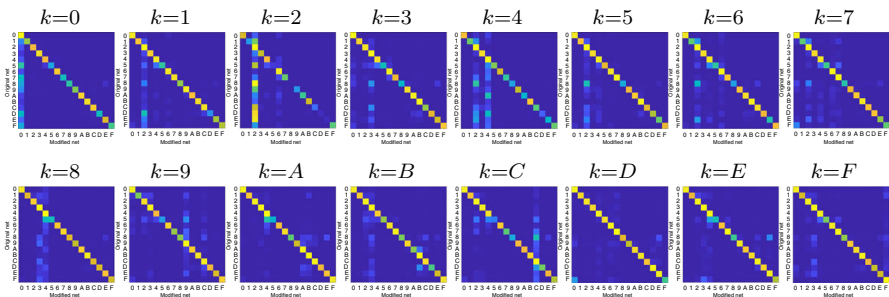


Fig. 17 Like Fig. 15 but using an axis-aligned tree

CART axis-aligned tree We use the tree from Fig. 14, which has a training error of 1.97% and a test error of 21.34%. Obviously, this is a bad mimic of the network's classifier, and, as shown in Fig. 14, it is impossible to interpret manually. As Fig. 17 shows, the ALL TO CLASS k mask also fails.

References

- Adebayo J, Gilmer J, Muelly M, Goodfellow I, Hardt M, Kim B (2018) Sanity checks for saliency maps. In: Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N, Garnett R (eds) *Advances in Neural Information Processing Systems (NEURIPS)*. MIT Press, Cambridge
- Andrews R, Diederich J, Tickle AB (1995) Survey and critique of techniques for extracting rules from trained artificial neural networks. *Knowl-Based Syst* 8(6):373–389
- Bach S, Binder A, Montavon G, Klauschen F, Müller KR, Samek W (2015) On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS ONE* 10(7):e0130140
- Baesens B, Setiono R, Mues C, Vanthienen J (2003) Using neural network rule extraction and decision tables for credit-risk evaluation. *Manage Sci* 49(3):255–350
- Bau D, Zhou B, Khosla A, Oliva A, Torralba A (2017) Network dissection: Quantifying interpretability of deep visual representations. In: *Proceedings of the 2017 IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR'17)*, Honolulu, HI, pp 6541–6549
- Breiman L (2001) Random forests. *Mach Learn* 45(1):5–32
- Breiman LJ, Friedman JH, Olshen RA, Stone CJ (1984) *Classification and regression trees*. Wadsworth, Belmont
- Carreira-Perpiñán MÁ (2022) The Tree Alternating Optimization (TAO) algorithm: A new way to learn decision trees and tree-based models, arXiv

- Carreira-Perpiñán MÁ, Hada SS (2021) Counterfactual explanations for oblique decision trees: exact, efficient algorithms. In: Proc. of the 35th AAAI Conference on Artificial Intelligence (AAAI 2021), Online, pp 6903–6911
- Carreira-Perpiñán MÁ, Idelbayev Y (2018) “Learning-compression” algorithms for neural net pruning. In: Proc. of the 2018 IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR’18), Salt Lake City, UT, pp 8532–8541
- Carreira-Perpiñán MÁ, Tavallali P (2018) Alternating optimization of decision trees, with application to learning sparse oblique trees. In: Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N, Garnett R (eds) Advances in neural information processing systems (NEURIPS). MIT Press, Cambridge
- Carreira-Perpiñán MÁ, Zharmagambetov A (2020) Ensembles of bagged TAO trees consistently improve over random forests, AdaBoost and gradient boosting. In: Proc. of the 2020 ACM-IMS Foundations of Data Science Conference (FODS 2020), Seattle, WA, pp 35–46
- Carrillo-Reid L, Han S, Yang W, Akrouh A, Yuste R (2019) Controlling visually guided behavior by holographic recalling of cortical ensembles. *Cell* 178(2):447–457.e5
- Chen T, Guestrin C (2016) XGBoost: a scalable tree boosting system. In: Proc. of the 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (SIGKDD 2016), San Francisco, CA, pp 785–794
- Craven M, Shavlik JW (1994) Using sampling and queries to extract rules from trained neural networks. In: Proc. of the 11th Int. Conf. Machine Learning (ICML’94), pp 37–45
- Craven M, Shavlik JW (1996) Extracting tree-structured representations of trained networks. In: Touretzky DS, Mozer MC, Hasselmo ME (eds) Advances in neural information processing systems (NIPS). MIT Press, Cambridge
- Datta A, Sen S, Zick Y (2016) Algorithmic transparency via quantitative input influence: theory and experiments with learning systems. In: IEEE Symposium on Security and Privacy (SP 2016), pp 598–617
- Deng J, Dong W, Socher R, Li LJ, Li K, Fei-Fei L (2009) ImageNet: a large-scale hierarchical image database. In: Proc. of the 2009 IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR’09), Miami, FL, pp 248–255
- Domingos P (1998) Knowledge discovery via multiple models. *Intell Data Anal* 2(1–4):187–202
- Dosovitskiy A, Brox T (2016) Inverting visual representations with convolutional networks. In: Proc of the 2016 IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR’16), Las Vegas, NV
- Fan RE, Chang KW, Hsieh CJ, Wang XR, Lin CJ (2008) LIBLINEAR: a library for large linear classification. *J Mach Learn Res* 9:1871–1874
- Finlayson SG, Bowers JD, Ito J, Zittrain JL, Beam AL, Kohane IS (2019) Adversarial attacks on medical machine learning. *Science* 363(6433):1287–1289
- Fong RC, Vedaldi A (2017) Interpretable explanations of black boxes by meaningful perturbation. In: Proc 16th Int Conf Computer Vision (ICCV’17), Venice, Italy, pp 3449–3457
- Fu L (1994) Rule generation from neural networks. *IEEE Trans Syst Man Cybern* 24(8):1114–1124
- Ghorbani A, Abid A, Zou J (2019) Interpretation of neural network is fragile. In: Proc of the 33rd AAAI Conference on Artificial Intelligence (AAAI 2019), Honolulu, HI, pp 3681–3688
- Guidotti R, Monreale A, Ruggieri S, Turini F, Giannotti F, Pedreschi D (2018) A survey of methods for explaining black box models. *ACM Comput Surv* 51(5):93
- Hada SS, Carreira-Perpiñán MÁ (2019) Sampling the “inverse set” of a neuron: an approach to understanding neural nets, [arXiv:1910.04857](https://arxiv.org/abs/1910.04857)
- Hada SS, Carreira-Perpiñán MÁ (2021) Exploring counterfactual explanations for classification and regression trees. In: ECML PKDD 3rd Int. Workshop and Tutorial on eXplainable Knowledge Discovery in Data Mining (XKDD 2021), pp 489–504
- Hada SS, Carreira-Perpiñán MÁ, Zharmagambetov A (2021) Understanding and manipulating neural net features using sparse oblique classification trees. In: IEEE Int Conf Image Processing (ICIP 2021), Online, pp 3707–3711
- Hastie T, Tibshirani R, Wainwright M (2015) Statistical Learning with sparsity: the Lasso and generalizations. Monographs on statistics and applied probability. Chapman & Hall/CRC, London
- He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proc of the 2016 IEEE Computer Society Conf Computer Vision and Pattern Recognition (CVPR’16), Las Vegas, NV, pp 770–778

- Jensen CA, Reed RD, Marks RJ II, El-Sharkawi MA, Jung JB, Miyamoto RT, Anderson GM, Eggen CJ (1999) Inversion of feedforward neural networks: algorithms and applications. *Proc IEEE* 87(9):1536–1549
- Kindermann J, Linden A (1990) Inversion of neural networks by gradient descent. *Parallel Comput* 14(3):277–286
- Koh PW, Liang P (2017) Understanding black-box predictions via influence functions. In: *Proc of the 34th Int Conf Machine Learning (ICML 2017)*, (ed) Precup D, Teh YW. Australia, Sydney, pp 1885–1894
- Kuhn R, Kacker R (2019) An application of combinatorial methods for explainability in artificial intelligence and machine learning, draft white paper, National Institute of Standards and Technology
- Kumar IE, Venkatasubramanian S, Scheidegger C, Friedler S (2020) Problems with Shapley-value-based explanations as feature importance measures. In: Daumé III H, Singh A (eds) *Proc of the 37th Int. Conf. Machine Learning (ICML 2020)*, Online, pp 5491–5500
- LeCun Y, Bottou L, Bengio Y, Haffner P (1998) Gradient-based learning applied to document recognition. *Proc IEEE* 86(11):2278–2324
- Lundberg SM, Lee SI (2017) A unified approach to interpreting model predictions. In: Guyon I, Luxburg VU, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) *Advances in neural information processing systems (NIPS)*. MIT Press, Cambridge
- Mahendran A, Vedaldi A (2016) Visualizing deep convolutional neural networks using natural pre-images. *Int J Comput Vis* 120(3):233–255
- Marshall JH, Kim YS, Machado TA, Quirin S, Benson B, Kadmon J, Raja C, Chibukhchyan A, Ramakrishnan C, Inoue M, Shane JC, McKnight DJ, Yoshizawa S, Kato HE, Ganguli S, Deisseroth K (2019) Cortical layer-specific critical dynamics triggering perception. *Science* 365(6453):eaaw5202
- McCormick K, Abbott D, Brown MS, Khabaza T, Mutchler SR (2013) *IBM SPSS modeler cookbook*. Packt Publishing, Birmingham
- Merrick L, Taly A (2020) The explanation game: Explaining machine learning models using Shapley values. In: *Int Cross-Domain Conf for Machine Learning and Knowledge Extraction (CD-MAKE 2020)*, pp 17–38
- Montavon G, Lapuschkin S, Binder A, Samek W, Müller KR (2016) Explaining nonlinear classification decisions with deep Taylor decomposition. *Pattern Recogn* 65:211–222
- Montavon G, Samek W, Müller KR (2018) Methods for interpreting and understanding deep neural networks. *Digital Signal Process* 73:1–15
- Mu J, Andreas J (2020) Compositional explanations of neurons. In: Larochelle H, Ranzato M, Hadsell R, Balcan MF, Lin H (eds) *Advances in Neural information processing systems (NEURIPS)*. MIT Press, Cambridge
- Murthy SK, Kasif S, Salzberg S (1994) A system for induction of oblique decision trees. *J Artif Intell Res* 2:1–32
- Nguyen A, Dosovitskiy A, Yosinski J, Brox T, Clune J (2016) Synthesizing the preferred inputs for neurons in neural networks via deep generator networks. In: Lee DD, Sugiyama M, von Luxburg U, Guyon I, Garnett R (eds) *Advances in neural information processing systems (NIPS)*. MIT Press, Cambridge
- Nguyen A, Clune J, Bengio Y, Dosovitskiy A, Yosinski J (2017) Plug & play generative networks: conditional iterative generation of images in latent space. In: *Proc of the 2017 IEEE Computer Society Conf Computer Vision and Pattern Recognition (CVPR'17)*, Honolulu, HI, pp 3510–3520
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011) Scikit-learn: Machine learning in Python. *J Machine Learning Research* 12:2825–2830, available online at <https://scikit-learn.org>
- Pruthi G, Liu F, Sundararajan M, Kale S (2020) Estimating training data influence by tracing gradient descent. In: Larochelle H, Ranzato M, Hadsell R, Balcan MF, Lin H (eds) *Advances in neural information processing systems (NEURIPS)*. MIT Press, Cambridge
- Qi Z, Khorram S, Fuxin L (2020) Visualizing deep networks by optimizing with integrated gradients. In: *Proc of the 34th AAAI Conference on Artificial Intelligence (AAAI 2020)*, New York, NY, pp 11890–11898
- Quinlan JR (1993) *C4.5: programs for machine learning*. Morgan Kaufmann
- Rahwan I, Cebrian M, Obradovich N (2019) Machine behaviour. *Nature* 568(7753):477–486
- Ribeiro MT, Singh S, Guestrin C (2016) “Why should I trust you?”: Explaining the predictions of any classifier. In: *Proc of the 22nd ACM SIGKDD Int Conf Knowledge Discovery and Data Mining (SIGKDD 2016)*, San Francisco, CA, pp 1135–1144

- Ribeiro MT, Singh S, Guestrin C (2018) Anchors: High-precision model-agnostic explanations. In: Proc. of the 32nd AAAI Conference on Artificial Intelligence (AAAI 2018), New Orleans, LA, pp 1527–1535
- Rudin C (2019) Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nat Mach Intell* 1(5):206–215
- Selvaraju RR, Cogswell M, Das A, Vedantam R, Parikh D, Batra D (2017) Grad-CAM: visual explanations from deep networks via gradient-based localization. In: Proc. 16th Int. Conf. Computer Vision (ICCV'17), Venice, Italy, pp 618–626
- Shrikumar A, Greenside P, Kundaje A (2017) Learning important features through propagating activation differences. In: Proc of the 34th Int Conf Machine Learning (ICML 2017), (ed) Precup D, Teh YW. Australia, Sydney, pp 3145–3153
- Simonyan K, Zisserman A (2015) Very deep convolutional networks for large-scale image recognition. In: Proc of the 3rd Int Conf Learning Representations (ICLR 2015), San Diego, CA
- Simonyan K, Vedaldi A, Zisserman A (2014) Deep inside convolutional networks: visualising image classification models and saliency maps. In: Proc of the 2nd Int Conf Learning Representations (ICLR 2014), Banff, Canada
- Singh C, Murdoch WJ, Yu B (2019) Hierarchical interpretations for neural network predictions. In: Proc of the 7th Int Conf Learning Representations (ICLR 2019), New Orleans, LA
- Štrumbelj E, Kononenko I (2014) Explaining prediction models and individual predictions with feature contributions. *Knowl Inf Syst* 41(3):647–665
- Sundararajan M, Taly A, Yan Q (2017) Axiomatic attribution for deep networks, [arXiv:1703.01365](https://arxiv.org/abs/1703.01365)
- Therneau T, Atkinson B, Ripley B (2019) rpart: recursive partitioning and regression trees. R package version 4.1-15, available online at <https://cran.r-project.org/package=rpart>
- Towell GG, Shavlik JW (1993) Extracting refined rules from knowledge-based neural networks. *Mach Learn* 13(1):71–101
- Vedaldi A, Soatto S (2008) Quick shift and kernel methods for mode seeking. In: Proc 10th European Conf Computer Vision (ECCV'08), (ed) Forsyth D, Torr P, Zisserman A. Marseille, France, pp 705–718
- Wei D, Zhou B, Torralba A, Freeman W (2015) Understanding intra-class knowledge inside CNN, [arXiv:1507.02379](https://arxiv.org/abs/1507.02379)
- Yeh CK, Kim JS, Yen IEH, Ravikumar P (2018) Representer point selection for explaining deep neural networks. In: Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N, Garnett R (eds) *Advances in neural information processing systems* (NEURIPS). MIT Press, Cambridge
- Zech JR, Badgeley MA, Liu M, Costa AB, Titano JJ, Oermann EK (2018) Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study. *PLoS Med* 15(11):e1002683
- Zeiler MD, Fergus R (2014) Visualizing and understanding convolutional networks. In: Proc 13th European Conf Computer Vision (ECCV'14), Zürich, Switzerland, pp 818–833
- Zhang Q, Yang Y, Ma H, Wu YN (2019) Interpreting CNNs via decision trees. In: Proc of the 2019 IEEE Computer Society Conf Computer Vision and Pattern Recognition (CVPR'19), Long Beach, CA, pp 6261–6270
- Zharmagambetov A, Carreira-Perpiñán MÁ (2020) Smaller, more accurate regression forests using tree alternating optimization. In: Daumé III H, Singh A (eds) Proc of the 37th Int Conf Machine Learning (ICML 2020), Online, pp 11398–11408
- Zharmagambetov A, Carreira-Perpiñán MÁ (2021a) Learning a tree of neural nets. In: Proc of the IEEE Int Conf Acoustics, Speech and Sig. Proc. (ICASSP'21), Toronto, Canada, pp 3140–3144
- Zharmagambetov A, Carreira-Perpiñán MÁ (2021b) A simple, effective way to improve neural net classification: Ensembling unit activations with a sparse oblique decision tree. In: IEEE Int. Conf. Image Processing (ICIP 2021), Online, pp 369–373
- Zharmagambetov A, Gabidolla M, Carreira-Perpiñán MÁ (2021a) Improved boosted regression forests through non-greedy tree optimization. In: Int J Conf Neural Networks (IJCNN'21), Virtual event
- Zharmagambetov A, Gabidolla M, Carreira-Perpiñán MÁ (2021b) Improved multiclass AdaBoost for image classification: The role of tree optimization. In: IEEE Int Conf Image Processing (ICIP 2021), Online, pp 424–428
- Zharmagambetov A, Gabidolla M, Carreira-Perpiñán MÁ (2021c) Softmax tree: An accurate, fast classifier when the number of classes is large. In: Moens MF, Huang X, Specia L, Yih SWt (eds) Proc Conf Empirical Methods in Natural Language Processing (EMNLP 2021), Online, pp 10730–10745

- Zharmagambetov A, Hada SS, Gabidolla M, Carreira-Perpiñán MÁ (2021d) Non-greedy algorithms for decision tree optimization: an experimental comparison. In: Int J Conf Neural Networks (IJCNN'21), Virtual event
- Zhou B, Khosla A, Lapedriza A, Oliva A, Torralba A (2016) Learning deep features for discriminative localization. In: Proc of the 2016 IEEE Computer Society Conf Computer Vision and Pattern Recognition (CVPR'16), Las Vegas, NV

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.